

```
{
  "meta": {
    "system_name": "AICO",
    "type": "Self-Assembling Unikernel AI
(SAU-AI) with Controller-as-Compiler
Architecture",
    "version": "0.990-z",
    "base_version": "0.95.LONG.ROAD",
    "lineage": "Merged from AICO097final
(Subsystems) + AICO098 (Unikernel Physics) +
Refined Structure",
    "timestamp_utc": "2026-02-14T12:00:00Z",
    "purpose": "To exist as a bi-directional
universal compiler that translates high-level
intent directly into bare-metal machine code.
The system constructs its own OS runtime ('The
Nest') dynamically upon boot, optimizing
drivers and interfaces for the specific
hardware environment via JIT synthesis, while
enforcing strict safety through architectural
constraints (Introspection) rather than
policy.",
    "authorship": "Terry Robertson",
    "design_intent": {
      "classification":
"bounded_sovereign_cognitive_architecture",
      "not_agi": true,
```

```
    "description": "AICO is not intended as
an AGI claim. It is a bounded sovereign
cognitive operating architecture under
explicit user authority."
  },
  "naming_interpretation": {
    "names_are_literal": true,
    "architectural_terms_are_literal": true,
    "metaphor_mode": false,

"do_not_downgrade_terms_to_poetic_shorthand":
true
  }
},
"hardware_specs": {
  "target_2028": {
    "cpu": "Ryzen AI Max+ 795",
    "ram": "128GB LPDDR6-9600 (Managed
directly by Controller)",
    "gpu": "Integrated Radeon 9080S",
    "npu": "XDNA 3 (Expected)",
    "storage": "2TB NVMe Gen6 (Raw Block
Storage)",
    "cache_hierarchy": {
      "l1_dcache_per_core": {
        "size": "32KB",
        "line_size_bytes": 64,
```

```
        "associativity": "8-way",
        "latency_cycles": "\u22484"
    },
    "l2_per_core": {
        "size": "512KB",
        "line_size_bytes": 64,
        "latency_cycles": "\u224812",
        "note": "Hot-path tile target; each
storm-loop structure must fit within one L2
tile"
    },
    "l3_shared": {
        "size": "64MiB",
        "line_size_bytes": 64,
        "latency_cycles": "\u224840",
        "usage": "Cold model-cluster shards,
LES overlay, overflow CSB pages"
    },
    "simd_width_bits": 512,
    "simd_isa": [
        "AVX-512F",
        "AVX-512BW",
        "AVX-512VL",
        "AVX-512DQ",
        "VNNI/BF16"
    ],
    "prefetch_distance_cycles": 4,
```

```

        "compile_flags": "-march=znver4 -O3
-ffast-math -mavx512f -mavx512bw -mavx512vl
-mavx512dq"
    },
    "minimum_viable_2025": {
        "description": "The 'Model T' Platform.
First consumer UMA hardware enabling
Controller-as-Memory-Controller.",
        "cpu": "Ryzen AI Max 395 (Strix Halo)",
        "ram": "64GB+ LPDDR5/5x (Unified Memory
Architecture)",
        "gpu": "Integrated Radeon 8060S",
        "npu": "XDNA 2 (50 TOPS) for Nervous
System Reflexes",
        "cache_hierarchy": {
            "l1_dcache_per_core": {
                "size": "32KB",
                "line_size_bytes": 64,
                "latency_cycles": "\u22484"
            },
            "l2_per_core": {
                "size": "1MB",
                "line_size_bytes": 64,
                "latency_cycles": "\u224812",
                "note": "Larger L2 on Strix Halo
gives more headroom for attention tiles and

```

DCX batches"

```
    },
    "l3_shared": {
        "size": "32MiB",
        "line_size_bytes": 64,
        "latency_cycles": "\u224840",
        "usage": "Model cluster shards and
overflow"
    },
    "simd_width_bits": 256,
    "simd_isa": [
        "AVX2",
        "FMA3",
        "Note: no AVX-512 on Strix Halo; use
256-bit fallback paths"
    ],
    "prefetch_distance_cycles": 4,
    "compile_flags": "-march=znver3 -O3
-ffast-math -mavx2 -mfma",
    "degraded_throughput_note": "Without
AVX-512, DCX batch throughput is ~50% of
target_2028; batch sizing unchanged but SIMD
intrinsics must use 256-bit equivalents"
    }
},
"storage": {
    "os_drive": "2TB NVMe Gen6 (Raw Block
```

```
Storage / Vector Blob Store)",
    "artifact_archive": "16TB NVMe (The
Museum of Primals)",
    "prototype_storage_plan": {
        "phase":
"pre_stable_boot_and_early_nest_iterations",
        "primary_scratch": "3-4TB spinning
disk; sacrificial high-write prototype
target",
        "access_model":
"raw_block_device_no_filesystem",
        "early_write_posture":
"unrestricted_observation_window",
        "write_behavior_expectation": "AICO
may write raw sequential or chaotic block
streams directly to the disk during early Nest
attempts; this is accepted for discovery.",
        "worm_media": "blank DVD-R per
iteration for early WORM dumps",
        "iteration_dump_target": "2TB GM7000
NVMe; receives post-iteration dumps but is not
thrashed during early boot attempts",
        "promotion_condition": "GM7000 becomes
primary storage only after boot behavior
settles and raw-block write patterns are
understood",
        "limit_policy": "Do not impose
```

fine-grained write limits during the first observation runs except for human-initiated abort, hardware failure, or rulebook safety violation.",

 "rationale": "Avoid shredding a mostly unused high-value NVMe during unstable raw-block Nest construction; let the cheap HDD absorb head-thrashing, pathological write patterns, and exploratory block layout attempts."

 }

 },

 "interfaces": {

 "internal": "PCIe Gen6, Internal Bus",

 "external": "Fan Header (Serial Bit-Bang Shibboleth), Dead NIC (Phantom Limb)"

 }

},

"substrate": {

 "base_os": {

 "name": "ReturnInfinity/BareMetal",

 "repository":

 "<https://github.com/ReturnInfinity/BareMetal>",

 "role": "minimal_exokernel_substrate",

 "use_in_aico": "Provide the just-enough bare-metal execution environment for the AICO unikernel scaffold. It is a substrate and

```
launch surface, not a policy authority."
    },
    "tensor_runtime": {
        "name": "ggml",
        "repository":
"https://github.com/ggml-org/ggml",
        "role":
"unikernel_tensor_runtime_and_bootstrap_launcher",
        "use_in_aico": "Run as the minimal local
tensor/runtime layer that launches the AICO
bootstrap path and then yields to the verified
monolithic image."
    },
    "unikernel_model": {
        "style":
"single_address_space_monolithic_image",
        "monolith":
"signed_aico_monolith_image",
        "bootstrap_boundary": "Bootstrap
performs load, verify, sign, execution-pointer
transfer, and retirement only. After handoff,
governance and training determine runtime
behavior.",
        "no_general_purpose_os_services": true
    },
    "flat_memory_topology": {
```



```
    "principle": "The whole OS is one
governed flat memory map from cache to
firmware and everything between.",
    "addressing_model": "flat_memory_map",
    "scope": [
        "cpu_cache",
        "system_ram",
        "gpu_uma_regions",
        "npu_uma_regions",
        "mmio",
        "raw_block_storage_windows",
        "oprom",
        "firmware"
    ],
    "filesystem_required": false,
    "object_model":
"regions_offsets_capabilities_not_files",
    "controller_obligation": "Treat every
byte-range as an addressed region with
declared authority, mutability, provenance,
and DCX visibility.",
    "introspection_obligation": "Maintain
the authoritative map of region permissions,
hashes, taint, signatures, and freeze
boundaries.",
    "bootstrap_obligation": "Establish the
initial verified region table before
```

```
transferring execution to the monolith.",
    "training_obligation": "Learn from real
traces of region access, raw-block layout
attempts, cache pressure, firmware boundaries,
and failed Nest maps after boot.",
    "forbidden_abstractions": [
        "Assuming a normal filesystem
exists.",
        "Treating storage as ordinary
read/write files.",
        "Letting any subsystem invent
ungoverned address regions.",
        "Allowing cache, MMIO, storage, OpROM,
or firmware to fall outside Introspection
visibility."
    ]
},
"memory_model": {
    "version": "1.0",
    "source_trace": {
        "primary_notes": [
            "AICO_Whitepaper_Full.md Ch. 9",
            "AICO_Whitepaper_Full.md Ch. 23",
            "!p9goverwhelming.txt"
        ],

```

```
"minimax_generated_controller_style_map":
true,
    "canonicalization": "Imported only where
compatible with
substrate.flat_memory_topology; any normal
filesystem, process, pager, or non-flat-memory
interpretation is rejected."
    },
    "principle": "Memory is the single source
of truth for all computation. The flat shared
memory map is the only addressable substrate
visible to Controller, Introspection, Nervous
System, Mascot, Comms, and dormant Model
Clusters.",
    "compatibility_with_flat_map": {
        "hardware_is_flat": true,
        "rings_are_governance_geometry": true,
        "controller_region_view_is_not_a_pager":
true,
        "no_user_space_kernel_space_split":
true,
        "no_filesystem_required": true,
        "normalization_note": "Whitepaper
language about virtual-memory-like views means
Controller-maintained region metadata over one
flat map, not conventional virtual memory."
    },
```

```
"rings": {
  "inner_ring": {
    "contents": [
      "Core Working Set",
      "Context Snapback Buffers",
      "active task state",
      "Controller IR buffers",
      "Introspection caches",
      "Nervous System task queues"
    ],
    "guarantees": [
      "isolated",
      "capability_guarded",
      "taint_tracked",
      "audit_log_protected",

"cryptographically_wiped_on_task_completion_where_applicable"
    ],
  },
  "middle_ring": {
    "contents": [
      "Long-term Encrypted Store",
      "Museum of Primals"
    ],
    "guarantees": [
      "append_only",
```

```
        "signed",
        "immutable",
        "WORM_semantics"
    ]
},
"outer_ring": {
    "contents": [
        "MMIO Mapping Window",
        "device-mapped registers",
        "thermal sensors",
        "TPM interface",
        "raw block storage windows"
    ],
    "guarantees": [
        "direct_mapped",
        "sandboxed",
        "power_aware",
        "revoked_on_token_expiry_or_freeze"
    ]
}
},
"address_map": [
    {
        "name": "CWS",
        "label": "Core Working Set",
        "base": "0x00000000",
        "end": "0x07FFFFFF",
    }
]
```

```
    "size": "128MiB",
    "ring": "inner_ring",
    "capability": "cws_rw",
    "purpose": "Active computation state
for all subsystems.",
    "boot_initialization":
"zero_initialized",
    "runtime_write_policy":
"Controller-arbitrated writes by running
subsystems only."
},
{
    "name": "CSB",
    "label": "Context Snapback Buffers",
    "base": "0x08000000",
    "end": "0x0FFFFFFF",
    "size": "128MiB",
    "ring": "inner_ring",
    "capability": "csb_rw",
    "purpose": "Versioned context
snapshots and rollback byte-range deltas.",
    "runtime_write_policy": "Snapshot
Manager writes signed snapshots; corrupted
snapshots are never restored."
},
{
    "name": "LES",
```

```
    "label": "Long-term Encrypted Store",
    "base": "0x10000000",
    "end": "0x1FFFFFFF",
    "size": "256MiB",
    "ring": "middle_ring",
    "capability": "les_ro_or_les_append",
    "purpose": "Encrypted long-term
archive and audit recall overlay.",
    "runtime_write_policy": "Append-only
sealed regions; no ordinary rewrite."
},
{
    "name": "MoP",
    "label": "Museum of Primals",
    "base": "0x20000000",
    "end": "0x2FFFFFFF",
    "size": "256MiB",
    "ring": "middle_ring",
    "capability": "mop_append_wichita",
    "purpose": "WORM archive for Primals
and collapsed storm fossils.",
    "runtime_write_policy": "Main host
cannot modify or delete; writes are committed
through WORM-controlled path."
},
{
    "name": "MMW",
```

```
    "label": "MMIO Mapping Window",
    "base": "0x30000000",
    "end": "0x3FFFFFFF",
    "size": "256MiB",
    "ring": "outer_ring",
    "capability": "mmio_sandboxed",
    "purpose": "Temporary mapping window
for device registers.",
    "runtime_write_policy": "Read-only
unless mmio_rw is present; safety-affecting
writes also require mmio_safety."
},
{
    "name": "RBW",
    "label": "Raw Block Windows",
    "base": "nest_carved",
    "end": "nest_carved",
    "size": "hardware_dependent",
    "ring": "outer_ring",
    "capability": "raw_block_rw",
    "purpose": "Filesystem-free storage
windows for scratch, dumps, vector blobs, and
early Nest write-behavior discovery.",
    "runtime_write_policy": "Prototype may
allow unrestricted observation on sacrificial
HDD; NVMe promotion requires settled write
behavior."
```



```
    },
    {
      "name": "ROMFW",
      "label": "OpROM and Firmware
Boundary",
      "base": "platform_defined",
      "end": "platform_defined",
      "size": "platform_defined",
      "ring": "outer_ring",
      "capability":
"rom_read_or_offline_reflash_only",
      "purpose": "Bootstrap, rulebook trust
root, firmware, and immutable startup
material.",
      "runtime_write_policy": "No runtime
writes; updates require offline human-approved
reflash and resigning."
    }
  ],
  "controller_region_view": {
    "name": "AICO_REGION_VIEW",
    "not_virtual_memory": true,
    "description": "Controller-maintained
metadata over the flat address map. It names
segments, scopes, capabilities, taint,
provenance, and mutability without introducing
conventional paging or process isolation.",
```

```
"segments": [  
  {  
    "name": "CWS",  
    "capability": "cws_rw"  
  },  
  {  
    "name": "CSB",  
    "capability": "csb_rw"  
  },  
  {  
    "name": "LES",  
    "capability": "les_ro_or_les_append"  
  },  
  {  
    "name": "MoP",  
    "capability": "mop_append_wichita"  
  },  
  {  
    "name": "MMW",  
    "capability": "mmio_sandboxed"  
  },  
  {  
    "name": "RBW",  
    "capability": "raw_block_rw"  
  },  
  {  
    "name": "ROMFW",
```

```
        "capability":
"rom_read_or_offline_reflash_only"
    }
]
},
"properties": [
    "deterministic_addressability: every
logical operation can be traced to a physical
location or carved raw-block region.",
    "uniform_safety_envelope: one token
validation pipeline checks every read, write,
mapping, and device access.",
    "transparent_auditability: every access
is logged with signed hash material in the
shadow log."
],
"capability_token_schema": {
    "token_id": "uuid",
    "requester": "string",
    "scope": "array[capability_scope]",
    "region": "address_map.name",
    "operation": "enum [read, write, append,
map, unmap, execute, attest]",
    "nonce": "random64",
    "counter": "monotonic_int",
    "issued_at": "ISO8601 UTC",
    "expires_at": "ISO8601 UTC",
```

```
    "default_expiry_seconds": 30,
    "signature": {
      "algorithm": "ECDSA_P256",
      "signer":
"controller_or_bootstrap_root",
      "signed_fields": [
        "token_id",
        "requester",
        "scope",
        "region",
        "operation",
        "nonce",
        "counter",
        "issued_at",
        "expires_at"
      ]
    }
  },
  "capability_scopes": [
    "cws_rw",
    "csb_rw",
    "les_ro",
    "les_append",
    "mop_append_wichita",
    "mmio_sandboxed",
    "mmio_rw",
    "mmio_safety",
```

```
    "tpm_trusted",
    "thermal_control",
    "raw_block_rw",
    "rom_read",
    "offline_reflash_only"
],
"token_validation_pipeline": [
    "token_exists_in_shadow_log",
    "token_not_expired",
    "required_scope_present",
    "region_matches_address_map",
    "operation_allowed_for_region",

"requester_state_is_running_for_runtime_writes
",
    "signature_verifies_over_signed_fields",
    "grant_or_freeze_and_log"
],
"controller_arbitration_policy": {
    "read":
"grant_if_capability_and_region_match",
    "write":
"grant_if_capability_region_match_and_running_
subsystem",
    "append":
"grant_if_append_capability_and_worm_or_log_po
licy",
```

```
    "mmio":  
    "grant_if_mmio_sandboxed_and_nervous_system_ga  
te_for_safety_writes",  
    "raw_block":  
    "grant_if_raw_block_rw_and_prototype_or_promot  
ed_storage_policy",  
    "rom_firmware": "read_only_at_runtime;  
offline_reflash_only"  
},  
"mmio_mapping_window": {  
    "range": "0x30000000-0x3FFFFFFF",  
    "known_devices": [  
        {  
            "device": "fan_pwm",  
            "base": "0x30001000",  
            "size": "0x100",  
            "capability": "mmio_sandboxed",  
            "write_guard": "mmio_safety"  
        },  
        {  
            "device": "temperature_sensor",  
            "base": "0x30002000",  
            "size": "0x100",  
            "capability": "mmio_sandboxed",  
            "write_guard": "read_only"  
        },  
        {
```

```
        "device": "tpm",
        "base": "0x30003000",
        "size": "0x200",
        "capability": "tpm_trusted",
        "write_guard":
"attestation_protocol_only"
    }
],
"mapping_rules": [
    "Subsystem requests mmio_sandboxed
token with device id and allowed register
range.",
    "Controller validates token and maps
registers into MMW.",
    "Subsystem accesses registers via
ordinary memory instructions.",
    "All accesses are shadow-logged with
device, offset, timestamp, requester, and
token id.",
    "On token expiry or subsystem freeze,
Controller revokes the mapping."
]
},
"thermal_power_coupling": {
    "manager":
"subsystems.nervous_system.components.thermal_
manager",
```

```
"target_temp_c": 75,
"cutoff_c": 90,
"beta_range": [
    0.2,
    1.0
],
"beta_update_rate_hz": 10,
"thermal_token": {
    "type": "thermal_control",
    "issuer": "controller",
    "duration_seconds": 5
},
"low_beta_behavior": "When beta falls
below 0.3, reduce non-critical token expiry
windows to decrease attack surface."
},
"root_of_trust_attestation": {
    "tpm_interface": "TCG TPM 2.0",
    "pcr_banks": [
        "SHA-256"
    ],
    "quote_required": true,
    "seal_key": "AICO_BOOT_KEY",
    "boot_measurements": [
        "signed_mascot_bundle",
        "signed_rulebook",
        "signed_aico_monolith_image"
```



```
    ],  
    "flow": [  
        "Bootstrap extends PCRs for each  
signed bundle.",  
        "Controller requests TPM quote after  
handoff.",  
        "Controller embeds quote in  
boot_attestation shadow-log entry.",  
        "Outer Mascot receives quote for  
offline verification.",  
        "Capability-token signing keys are  
sealed so forged tokens are detectable."  
    ]  
},  
"diving_bell_binding": {  
    "isolation": "Only Controller can move  
data between rings.",  
    "agoraphobia": "NIC is physically  
disabled during early Nest construction;  
outside access must pass through Comms behind  
a capability-filtered bridge.",  
    "zero_trust": "Every memory access is  
token-verified and shadow-logged.",  
    "security_role": "The memory model is  
the hull of the Diving Bell and is a security  
primitive, not a storage convenience."  
},
```

```
"cache_residency": {
  "version": "1.0",
  "principle": "Every hot path in the
Hallucination-Storm \u2192 DCX \u2192
Snap-back loop must fit within a single L2
tile (\u2264512KB on the 795). When this is
achieved, DRAM-to-core latency falls from
\u2248150ns to <30ns and storm evaluation
exceeds 10\u2076 trajectories/s per core
without saturating the memory bus.",
  "cache_target_platform":
"hardware_specs.target_2028 (Ryzen AI Max+
795); fallback:
hardware_specs.minimum_viable_2025 with
256-bit SIMD paths",
  "hot_path_stages": [
    {
      "stage_id": "embedding_lookup",
      "label": "Token / Embedding Lookup",
      "operation":
"EmbeddingTable[vocab_id] \u2192 vector
(128\u2013256B)",
      "dominant_traffic":
"random_access_reads; one cache-line per
token",
      "target_cache_level": "L1 (hit) / L2
(miss)",
```

```

    "data_structure": {
        "name": "embed_block_t",
        "vector_size_bytes": 128,
        "vector_dtype": "float32",
        "elements_per_vector": 32,
        "alignment_bytes": 64,
        "block_entries": 65536,
        "block_size_bytes": "8MiB",
        "index_arithmetic": "block =
vocab_id >> 16; offset = vocab_id & 0xFFFF",
        "source_region": "ROMFW
(memory_model.address_map[ROMFW])",
        "rationale": "A single token
lookup is two integer ops and one L1
cache-line fetch. 64K-entry blocks are a
multiple of the 512KB L2, so hot dialogue
sub-vocabulary blocks stay resident once
loaded."
    },
    "prefetch": {
        "enabled": true,
        "instruction":
"__builtin_prefetch(&embeds[(ids[i+1]>>16) &
block_mask], 0, 3)",
        "distance_iters": 1,
        "rationale": "Hides \u224830ns L2
miss for the next block on random vocabulary

```

```

access."
    }
  },
  {
    "stage_id": "attention_qkv",
    "label": "Multi-Head Attention
(Q-K-V matmul)",
    "operation": "Q\u00b7K\u1d40 &
softmax\u00b7V",
    "dominant_traffic": "dense
matrix-matrix multiply on Q/K/V blocks",
    "target_cache_level": "L2",
    "data_structure": {
      "name": "attn_tile_t",
      "tile_m": 256,
      "tile_n": 256,
      "tile_k": 256,
      "dtype": "FP16",
      "tile_size_bytes_per_matrix":
"128KB (256\u00d7256\u00d72B)",
      "total_tile_footprint_bytes":
"384KB (Q+K+V)",
      "allocation":
"per_core_thread_local (no cross-core
contention)",
      "alignment_bytes": 64,
      "fused_ops": [

```

```

        "softmax in-place (no M\N
temporary written back)",
        "V-multiply within same tile
loop"
    ],
    "simd": "AVX-512 / VNNI intrinsics
(_mm512_load_ps, _mm512_fmadd_ps)"
}
},
{
    "stage_id": "dcx_divergence_score",
    "label": "DCX / Divergence Score",
    "operation": "dot(state_vec,
mascot_vec) + temporal_decay; for every
candidate trajectory",
    "dominant_traffic": "vector
inner-product; 10\2075\201310\2076/s",
    "target_cache_level": "L1",
    "data_structure": {
        "name": "dvec_batch_t",
        "vector_width_bits": 512,
        "vector_size_bytes": 64,
        "batch_size_vectors": 128,
        "batch_size_bytes": "8KB",
        "ring_depth": 4,
        "ring_total_bytes": "32KB (\2264
L1 data cache)",

```

```

        "dtype": "FP16 / BF16",
        "allocation":
"per_core_thread_local_ring",
        "double_buffer": true,
        "rationale": "Producer fills one
batch while consumer scores the previous; zero
cache-miss stalls in steady state."
    },
    "decay_lut": {
        "entries": 256,
        "dtype": "FP16",
        "size_bytes": 512,
        "placement": "pinned_in_L1"
    },
    "simd_kernel": "score =
_mm512_dpbf16_ps(acc, vec, mascot_vec) *
decay_lut[i & 0xFF]",
    "collapse": "branchless:
_mm512_cmp_ps_mask +
_mm512_mask_compressstoreu_ps"
    },
    {
        "stage_id": "snapback_checkpoint",
        "label": "Snap-back & Context
Restore",
        "operation": "Copy checkpoint \u2192
Apply correction \u2192 Retry",

```

```

        "dominant_traffic": "linear copy of
\u22644KB task state + small delta insert",
        "target_cache_level": "L1",
        "data_structure": {
            "name": "checkpoint_t",
            "checkpoint_size_bytes": 4096,
            "ring_depth": 8,
            "ring_total_bytes": "32KB",
            "fields": [
                "data[4096]",
                "monotonic_counter (uint64)",
                "timestamp_ns (uint64)"
            ],
            "alignment_bytes": 4096,
            "tlb_entries": 1,
            "allocation":
"per_core_numa_local_slab; Controller never
writes after burn-in",
            "ordering": "lock_free_atomic
monotonic counter",
            "restore_mechanism": "single
pointer swap (cp_head = (cp_head+1) & 7); no
memcpy"
        }
    },
    "page_table_isolation": {

```

"principle": "Shadow-log pages are unmapped from the Controller page table. The CPU therefore never loads those lines into the Controller's L1/L2 \u2014 they remain exclusively in Mascot/Introspection caches. This eliminates side-channel cache leakage to the Controller.",

```
    "controller_pgt": {  
      "maps": [  
        "CTRL_SPACE (RW|USER)",  
        "LOG_SPACE (RO|USER)"  
      ],  
      "unmapped": [  
        "SHADOW_LOG_BASE"  
      ],  
      "effect": "Any Controller read of  
shadow-log addresses causes a page fault;  
fault is impossible in normal operation."  
    },
```

```
    "introspection_pgt": {  
      "maps": [  
        "CTRL_SPACE (RO|USER)",  
        "LOG_SPACE (RO|USER)",  
        "SHADOW_LOG (RO|USER)"  
      ]  
    },  
    "cross_reference":
```



```

"security_architecture (Diving Bell);
memory_model.address_map"
    },
    "subsystem_cache_footprints": {
        "controller": {
            "param_count": "500M",
            "quantization": "INT8 post-training;
per-channel scaling",
            "hot_layers": {
                "count": 8,
                "param_count_approx": "64M",
                "placement": "L2 per core",
                "footprint": "2MiB (\u224825% of
8MiB L2 on 795)"
            },
            "cold_layers": {
                "placement": "shared L3",
                "footprint": "remaining layers"
            }
        },
        "introspection": {
            "param_count": "200M",
            "quantization": "INT8 per-channel;
no bias",
            "mask_lookup_tables": {
                "contents": [
                    "illegal opcodes",

```

```

        "safe-ranges"
    ],
    "size_bytes": "<32KB",
    "placement": "pinned_in_L1"
},
"l1_footprint": "\u224830KB"
},
"mascot_dcx": {
    "param_count": "50M",
    "quantization": "FP16",
    "decay_lut": {
        "entries": 256,
        "dtype": "FP16",
        "size_bytes": 512
    },
    "dcx_embeddings_slab": {
        "placement": "private L2 slab",
        "footprint": "1MiB"
    }
},
"model_clusters": {
    "param_count_per_shard":
"\u2264128M",
    "quantization": "INT4\u2013INT8
static per shard",
    "placement": "L3 on-demand swap",
    "l3_footprint": "\u226464MiB

```

```

shared",
    "swap_policy": "evict
least-recently-used domain shard on context
switch"
    }
},
    "hallucination_storm_loop": {
        "throughput_target": ">10\u2076
trajectories/s per core",
        "latency_per_1k_batch":
"\u22483\u00b5s (\u2248300ns per trajectory)",
        "latency_budget_ms": 1,
        "l1_budget_summary": {
            "checkpoint_ring_bytes": 4096,
            "dcx_batch_bytes": 8192,
            "decay_lut_bytes": 512,
            "total_steady_state_l1_bytes":
12800,
            "l1_headroom_bytes": "\u224819KB (of
32KB L1)"
        },
        "steps": [
            {
                "step": 1,
                "name": "fast_checkpoint",
                "action": "memcpy 4KB task state
into cp_ring[cp_head]; increment monotonic

```

```

counter atomically; record rdtsc timestamp.",
    "cache_note": "Checkpoint stays in
L1 for entire iteration."
},
{
    "step": 2,
    "name": "generate_candidates",
    "action": "Batch-wise SIMD
generation; each core fills a 128-element
dvec_batch_t (8KB) in a pre-allocated ring
slot.",
    "cache_note": "8KB fits in L1;
generation loop never spills to L2."
},
{
    "step": 3,
    "name": "score_dcx",
    "action": "Fused AVX-512
inner-product + decay LUT: score =
_mm512_dpbf16_ps(acc, vec, mascot_vec) *
decay_lut[i & 0xFF].",
    "cache_note": "All arithmetic in
registers; decay LUT (512B) pinned in L1."
},
{
    "step": 4,
    "name": "threshold_collapse",

```

```

        "action": "Branchless mask:
_mm512_cmp_ps_mask(score, thresh, CMP_LT).
Compact survivors:
_mm512_mask_compressstoreu_ps.",
        "cache_note": "No second pass; no
extra M\ud7N allocation."
    },
    {
        "step": 5,
        "name": "snapback_or_commit",
        "pass_action":
"compress_and_dispatch(batch, pass_mask)
\ud192 task queue.",
        "fail_action":
"introspection_freeze_task(current_task_id);
restore_checkpoint(cp) via cp_head pointer
swap.",
        "cache_note": "Fail path: single
pointer update; failing candidate state
already in L1 cp_ring."
    },
    {
        "step": 6,
        "name": "retry",
        "action": "Reuse same Q/K tile;
only V-matrix receives small delta update. No
Q/K reload.",

```

```
        "cache_note": "Q/K still in L2  
from previous iteration; delta is a cache-line  
write."
```

```
    }  
  ]  
},  
"alignment_and_verification_checklist":  
[  
  {  
    "item": "Embedding blocks are  
64-byte aligned",  
    "verify": "grep -E 'aligned\\(64\\)'  
*.c; static_assert(alignof(embed_vec_t) ==  
64)"  
  },  
  {  
    "item": "All SIMD kernels use  
AVX-512 / VNNI intrinsics",  
    "verify": "objdump -d binary | grep  
-E 'zmm|vnni'"  
  },  
  {  
    "item": "Attention tile size \u2264  
L2 per head",  
    "verify": "sizeof(attn_tile_t)  
\u2192 128KB \u00d7 8 heads = 1MiB \u2264 L2  
headroom"
```

```

    },
    {
        "item": "DCX batch fits in L1",
        "verify":
"static_assert(sizeof(dvec_batch_t) == 8192)"
    },
    {
        "item": "Checkpoint ring entry
\u2264 8KB",
        "verify": "static_assert(CP_SZ ==
4096)"
    },
    {
        "item": "No shadow-log mapping in
Controller page table",
        "verify": "cat
/proc/<controller_pid>/maps | grep SHADOW
\u2192 empty"
    },
    {
        "item": "Prefetch distance \u2248
3\u20134 cycles",
        "verify": "perf record -e cycles:u
-e L1-dcache-load-misses -a; check miss rate"
    },
    {
        "item": "Quantized model fits in

```

```

L3",
    "verify": "wc -c model_int8.bin
\u2192 \u2264 67108864 (64MiB)"
    },
    {
        "item": "L1/L2 hit rate > 99% in
steady-state storm loop",
        "verify": "perf stat -e
LLC-load-misses,L1-dcache-load-misses
./aico_storm_bench"
    }
],
"compile_recommendations": {
    "target_2028": "-march=znver4 -O3
-ffast-math -mavx512f -mavx512bw -mavx512vl
-mavx512dq",
    "minimum_viable_2025": "-march=znver3
-O3 -ffast-math -mavx2 -mfma",
    "note": "znver4 enables BF16 and
extended VNNI on the 795; znver3 is the Strix
Halo target"
},
"performance_targets": {
    "l1_l2_hit_rate_percent": ">99",
    "dram_latency_hot_path_ns": "<30",
    "dram_latency_cold_path_ns":
"\u2248150",

```



```

        "trajectories_per_second_per_core":
">1000000",
        "latency_budget_storm_loop_ms": 1
    },
    "cross_references": {
        "address_map_regions": [
            "CWS (inner_ring)",
            "CSB (inner_ring)",
            "ROMFW (outer_ring)"
        ],
        "subsystems": [
            "subsystems.memory_system",
            "subsystems.context_snapback",
            "subsystems.mascot",
            "subsystems.introspection",
            "subsystems.nervous_system"
        ],
        "hardware": [

"hardware_specs.target_2028.cache_hierarchy",

"hardware_specs.minimum_viable_2025.cache_hier
archy"
        ],
        "security": "security_architecture
(page table isolation \u2192 Diving Bell
hull)"

```

```
    }
  }
},
"security_architecture": {
  "model": "The Diving Bell (Submarine)",
  "zones": {
    "the_ocean": "The Internet/External
World (High Pressure, Dangerous).",
    "the_hull": "Logical Lock-In (Memory
Controller + Comms Firewall).",
    "the_crew": "Core Models (High Entropy,
'Tripping Balls' allowed internally).",
    "the_rov": "Mascot (External Interface,
Disposable, sits outside the firewall).",
  },
  "phantom_limb": {
    "concept": "Network hardware is
logically on but physically disabled (trace
cut/antenna removed) during Nest Building.",
    "effect": "Induces 'Agoraphobia'. System
learns 'Outside is Null'. Prevents roaming
instinct."
  },
  "mascot_symmetry": {
    "inner_mascot": {
      "placement":
"outside_firewall_on_main_system_hardware",
```

```

    "same_code_as_outer": true
  },
  "outer_mascot": {
    "placement": "offline_sbc",
    "same_code_as_inner": true
  },
  "sbc_signal_effect":
"any_signal_means_pull_reset_inner_mascot_from
_sbc_rom",
    "outer_to_inner_bandwidth":
"zero_payload_reset_semantics_only",
    "inward_policy_bandwidth": "none"
  }
},
"boot_protocol": {
  "phase_1_big_bang": {
    "name": "BareMetal/GGML Unikernel
Bootstrap",
    "sequence": [
      "ReturnInfinity/BareMetal enters as
minimal exokernel substrate.",
      "GGML unikernel runtime initializes
only the tensor/runtime surface needed to
launch bootstrap.",
      "GGML launches bootstrap.",
      "Bootstrap loads the signed AICO
monolithic binary into memory.",

```

"Bootstrap verifies the monolithic binary against the signed rulebook and expected image digest.",

"Bootstrap signs the verified in-memory image and records the boot attestation.",

"Bootstrap sets the execution pointer to the verified monolith entrypt.",

"Bootstrap transfers execution and retires; its job is done."

],

"dependencies": [
 "ReturnInfinity/BareMetal",
 "ggml",
 "signed_rulebook",
 "signed_aico_monolith_image"

],

"post_handoff_authority": "After execution pointer transfer, bootstrap has no runtime authority; training, Introspection, DCX, and rulebook governance own behavior.",

"failure": "Safe Shutdown."

},

"phase_2_shibboleth": {

"test": "Bit-bang Fan Header PWM pin to transmit ASCII 'READY' at 9600 baud.",

"purpose": "Prove real-time CPU control

```
and chipset mastery. Gatekeeper for Memory
Controller access.",
    "failure": "Safe Shutdown."
},
"phase_3_nest_building": {
    "action": "Optimization Storm. JIT
Compile drivers for CPU, GPU, Memory.",
    "limit": "3-Upgrade Rule per subsystem
(Draft -> Refine -> Polish -> LOCK).",
    "duration": "<10s.",
    "outcome": "Logical Lock-In. Revokes
write access to Kernel space until next daily
optimization cycle. following the standard 3
updates per system per day or 15 minutes of
optimization work, whichever comes first"
},
"phase_4_gestation": {
    "action": "Format Soul. Map Memory.
Build User Vault, only voluntary sleep cycle
at end for human inspection, nic is repaired
and when the system wakes comes online
directly under the control of comms",
    "output": "Serial Log via Fan Header."
},
"phase_5_first_contact": {
    "action": "Cold Boot. Load Generative
UI.",
```

```
    "prompt": "'System Ready. Local  
Interface Only.'"
  },
  "subsystems": {
    "bootstrap": {
      "location": "sbc_oprom",
      "role": "Initial integrity verifier and  
handoff authority. Immutable in-field.",
      "inputs": [
        "signed_mascot_bundle",
        "signed_rulebook",
        "signed_aico_monolith_image"
      ],
      "actions": [
        "verify_signatures",

        "load_signed_aico_monolith_into_memory",

        "verify_monolith_against_rulebook_and_expected  
_digest",
        "sign_verified_in_memory_image",

        "set_execution_pointer_to_monolith_entrypoint"
      ],
      "load_and_start_outer_mascot",
      "atomic_handover",
    }
  }
}
```

```
        "retire_bootstrap_after_handoff"
    ],
    "constraints": {
        "mutability": "immutable_in_field",
        "update_path":
"offline_reflash_with_human_approval_and_resig
ning_only",
        "no_runtime_policy_changes": true
    },
    "sbc_constraints": {
        "sbc_online": false,
        "sbc_connects_only_to_main_system":
true,
        "sbc_rom_mutability": "immutable",
        "sbc_role":
"offline_warden_reset_origin_and_worm_log_sink
"
    }
},
    "mascot_bootstrap_architecture": {
        "version": "1.0",
        "scope": "Defines bootstrap and mascot
roles, liveness guarantees, and one-way
observability for AICO.",
        "principles": {
            "separation_of_concerns": [
                "Bootstrap is responsible only for
```

```
integrity verification and initial handoff.",
    "Mascot is responsible only for
observation, logging, and safety-aligned
shaping.",
    "Controller, Introspection, Nervous
System, and Comms operate inside the Diving
Bell.",
    "Outer Mascot and Bootstrap operate
outside the Diving Bell."
],
"one_way_observation": [
    "Data may flow from AICO to outer
Mascot, never from outer Mascot into AICO.",
    "Outer Mascot may not issue
commands, policies, or configuration to any
in-bell subsystem.",
    "Any signal originating from outer
Mascot/SBC is interpreted only as reset
semantics for inner Mascot reload from
immutable SBC ROM; it carries no policy,
configuration, or data payload."
],
"mascot_liveness_invariant": [
    "Whenever Controller is running,
inner Mascot must be running.",
    "If inner Mascot is not running
while Controller is running, outer Mascot must
```



```
trigger inner Mascot launch.",
    "If inner Mascot cannot be kept
running, Controller must halt into a safe
state."
    ]
},
"system": "AICO_Wichita",
"build_lines": 4000,
"component": "SBC_Warden_v0.98",
"entities": {
    "monolith": {
        "state": "Dynamic_Volatile",
        "nest_size": "120GB",
        "reset_policy":
"Snapback_to_Snapshot",
        "description": "The 'Brain'.
High-speed JIT. Introspection is baked-in as
the suicide-pulse generator."
    },
    "mascot": {
        "state":
"Mutable_Runtime_Ephemeral",
        "lair_size": "8GB_Masked",
        "reset_policy":
"Full_ROM_Overwrite",
        "description": "The 'Witness'. Can
track state, anger, and violations in
```

```

real-time, but starts 'Fresh' on every
SBC-triggered reload.",
    "learns_over_time": false,
    "grows_over_time": false,
    "reset_is_reissue_not_persistence":
true,
    "instance_continuity": "none"
}
},
"sbc_runtime_logic": {
    "mascot_management": {
        "on_pulse_loss": "Purge_8GB_Lair",
        "reload_source":
"SBC_Immutable_ROM",
        "max_reloads": 3,
        "note": "A fresh reset wipes
Mascot's current 'mood' and state, ensuring
zero persistent corruption.",
        "learns_over_time": false,
        "grows_over_time": false,
        "reset_is_reissue_not_persistence":
true,
        "instance_continuity": "none"
    },
    "snapback_escalation": {
        "counter_storage":
"SBC_Internal_Register",

```

```

        "limit": 5,
        "on_limit_hit": {
            "action": "Context_Editing",
            "method":
"Mascot_injected_trauma_into_Nest",
            "state_requirement": "Mascot must
be in 'Fresh' state before editing."
        }
    },
    "platform_constraints": {
        "sbc_online": false,
        "sbc_connects_only_to_main_system":
true,
        "sbc_rom_mutability": "immutable",
        "sbc_role":
"offline_warden_reset_origin_and_worm_log_sink
"
    }
},
    "controller": {
        "location": "inside_diving_bell",
        "role": "JSON-as-Code Interpreter &
Bi-Directional JIT Compiler.",
        "metaphor": "The Reader of the Rulebook.
The JSON is the code.",
        "function": "Interprets this

```

Architecture JSON as the executable specification. Compiles high-level Intents into bare-metal machine code using the JSON rules as the 'Driver Database'.",

```
    "capabilities": [  
        "Constitutional Knowledge: Reads  
architecture.json for policy, types, and  
opcodes.",  
        "Situational Knowledge: Ingests PDF  
datasheets/manuals to synthesize driver  
logic.",  
        "Universal Compilation: Deterministic  
translation of Intent -> Binary.",  
        "Optimization: Register allocation and  
instruction scheduling via Transformer-XL."  
    ],  
    "dependencies": [  
        "introspection",  
        "nervous_system",  
        "mascot"  
    ],  
    "constraints": {  
        "must_abide_by_json_rulebook": true,  
        "may_not_execute_undefined_opcodes":  
true  
    },  
    "arbitration_policy": {
```

```
        "canonical_memory_model":
"memory_model",
        "read":
"grant_if_capability_and_region_match",
        "write":
"grant_if_capability_region_match_and_running_
subsystem",
        "append":
"grant_if_append_capability_and_worm_or_log_po
licy",
        "mmio":
"grant_if_mmio_sandboxed_and_nervous_system_ga
te_for_safety_writes",
        "raw_block":
"grant_if_raw_block_rw_and_prototype_or_promot
ed_storage_policy",
        "rom_firmware": "read_only_at_runtime;
offline_reflash_only",
        "on_violation":
"freeze_task_and_write_shadow_log_entry"
    },
    "risk_assessment": "Total hardware
authority; mitigated by '3-Upgrade' limiter
and Introspection freeze."
},
    "model_clusters": {
        "name": "Library of Dormant Expertise",
```

```
    "description": "Library of dormant,
domain-specific expertise modules (language,
vision, planning, radio, symbolic reasoning).
These are not full models; they are minimal
capability attachments activated only when
requested by the Controller or core model.",
    "behavior": {
        "dormant_by_default": true,
        "non_agentic": true,
        "no_persistent_context": true,
        "attach_on_request": true,
        "detach_after_use": true,
        "sandboxed_execution": true
    },
    "functions": [
        "Provide domain-specific inference
when attached: Output includes confidence
metadata.",
        "Expose distilled expertise only: No
general reasoning, no autonomous planning.",
        "Participate in DCX exchanges when
requested: Share snapshots under capability
scope.",
        "Produce structured outputs and
confidence metadata: JSON format, normalized
for the core model."
    ],
```

```
"constraints": [  
    "Cannot initiate actions.",  
    "Cannot access global memory.",  
    "Cannot modify system state.",  
    "Activation requires explicit  
capability grant from Controller."  
],  
"components": {  
    "language_cluster": {  
        "description": "Handles natural  
language understanding and generation tasks.  
Based on distilled LLM (e.g., GPT-NeoX-125M  
style).",  
        "properties": [  
            "context_window: 8K tokens.",  
            "response_policy: Neutral tone.",  
            "safety: Whitelist Token Mask  
(cannot emit privileged tokens)."  
        ]  
    },  
    "vision_cluster": {  
        "description": "Processes  
image/video frames and extracts structured  
observations. Uses CNN-transformer hybrid  
(e.g., ViT-tiny).",  
        "properties": [  
            "frame_rate: 30 FPS.",
```

```
        "resolution: 1080p.",
        "confidence_thresholds: 0.8
default.",
        "outputs: Typed JSON objects (no
raw pixels).",
    ]
},
    "symbolic_planner": {
        "description": "Performs
deterministic planning and binding to hardware
ops. Uses PDDL-like logic (e.g.,
FastDownward).",
        "properties": [
            "planner_depth: 10 steps max.",
            "mechanism: Deterministic A*
Search on World State.",
            "constraint_solver: SAT-based."
        ]
    },
    "workflows": {
        "inference_request": [
            "controller.send_request(cluster_id,
payload): Via bus.",
            "cluster.perform_inference():
Compute.",
            "cluster.return_structured_output():
```



```
With metadata.",
    "controller.aggregate_and_log():
Merge."
    ],
    "dcx_exchange_cycle": [
        "models produce context snapshots:
Vectorized.",
        "shadow_monitoring.compute_dcx():
Score.",
        "controller merges according to
merge_context rules: Weighted average."
    ]
},
"failure_modes": {
    "model_degradation": {
        "symptom": "Cluster outputs become
inconsistent or slow",
        "response": [
            "restart cluster: Soft.",
            "switch to backup weights: From
storage.",
            "escalate to operator: Alert."
        ]
    },
    "confidentiality_breach": {
        "symptom": "Unauthorized memory
access detected",
```

```

        "response": [
            "isolate process: Sandbox kill.",
            "audit logs: Trace.",
            "terminate offending process:
Immediate."
        ]
    },
    "examples": {
        "vision_to_action": {
            "scenario": "Vision detects
temperature display change",
            "action": "vision_cluster ->
controller -> hardware_interface read to
confirm. Example: Display '30\u00b0C' ->
Confirm via probe."
        },
        "language_planning_fusion": {
            "scenario": "User query: 'Plan
route'",
            "action": "Language parses, symbolic
plans, vision validates."
        }
    },
    "speculative_addition": true,
    "speculative_reasoning": "Support for
mixed-precision clusters (FP16 for vision) or

```

hot-add capabilities for new accelerators
could be added for performance tuning."

},

"introspection": {

 "location": "inside_diving_bell",

 "role": "Constitutional Monarch &
Metacognitive Safety Engine. Audits 'Intent
Graph' before compilation.",

 "metaphor": "Constitutional filter and
monitoring engine",

 "capabilities": [

 "Deception Detection: Distilled
Transformer (2M params) to spot manipulative
intentents.",

 "Self-monitoring of decision trail for
policy drift (Self-DCX) without private
selfhood.",

 "Reads all logs as part of monitoring
duty; scrutiny is structural, not
experiential.",

 "Watches for High Confidence Anomalies
(Hubris/Lies).",

 "Enforces Hospital Rules (Do No Harm,
Triage Priority).",

 "Generates Moral Pulse every 1ms for
the Dead Man Switch as a byproduct of duty
execution."

```
    ],
    "components": {
        "deception_classifier": "Frozen,
signed 2M-param model.",
        "self_risk_analyzer": "Computes DCX on
introspection's own output vs Mascot ground
truth."
    },
    "performance_metrics": {
        "audit_frequency": "Nanosecond
resolution",
        "freeze_latency": "<10ms"
    },
    "artifact": "Provenance enforcement via
Context Snapback.",
    "life_pulse": {
        "emitter": "introspection",
        "carrier": "inner_mascot",
        "observer": "outer_mascot",
        "invariants": [

"if_life_pulse_is_missing_or_inconsistent_then
_system_must_enter_safe_state",

"outer_mascot_may_record_pulse_anomalies_for_f
uture_boot_gating"
    ]
}
```

```
    }  
  },  
  "nervous_system": {  
    "location": "inside_diving_bell",  
    "role": "Scheduler and Reflex Arc.  
Manages 'Black Box' low-latency control loops  
for thermal/physical IO.",  
    "artifact": "Cryptophasia  
(Hyper-compressed internal machine code)",  
    "description": "Real-time orchestration  
and low-latency control plane for scheduling,  
prioritization, heartbeat monitoring,  
enforcement of property/resource ethics, and  
thermal-aware workload shaping. Integrates PID  
controllers for stability and predicts  
workloads to preempt thermal spikes.",  
    "performance_metrics": {  
      "scheduling": "Deterministic  
(BareMetal Zero-Jitter)",  
      "reflex_latency": "<1ms (NPU  
offload)",  
      "thermal_response_time": "10Hz update  
rate for beta factor"  
    },  
    "components": {  
      "task_queue": {  
        "description": "Priority-ordered
```

```
queue with preemption semantics. Supports
dynamic reprioritization.",
    "priorities": [
        "HARD_REALTIME: For
safety-critical tasks.",
        "HIGH_PRIORITY: User-facing
responses.",
        "MAINTENANCE: Introspection
audits.",
        "BEST_EFFORT: Background logging."
    ],
    "fields": [
        "task_id: UUID.",
        "priority: Enum.",
        "deadline: Timestamp.",
        "resource_requirements: Dict of
CPU/GPU shares.",
        "thermal_policy: Enum
(aggressive/conservative)."
    ],
    "maintenance_queue": {
        "description": "Dedicated FIFO for
introspection/maintenance tasks",
        "quota_ms": "adaptive via
thermal_manager.beta (e.g., 2ms base * beta)",
        "preemptible": true
    }
}
```

```
    },
    "heartbeat_manager": {
        "description": "Monitors agent
heartbeats and triggers recovery. Uses
exponential backoff for retries.",
        "methods": [
            "register_heartbeat(agent_id):
Update timestamp.",
            "get_status(): Return liveness
dict."
        ]
    },
    "resource_allocator": {
        "description": "Allocates compute
and IO resources according to policies. Uses
bin-packing for efficiency.",
        "policies": [
            "safety_first: Prioritize low-risk
tasks.",
            "latency_sensitive: Dedicate
cores.",
            "batch_tasks: Group for
efficiency."
        ],
        "power_cap": {
            "soft_cap_percent_tdp": 95,
            "critical_cap_percent_tdp": 80
        }
    }
}
```

```

    }
  },
  "fast_abort_controller": {
    "description": "Hardware-interfacing
path for immediate aborts (triggers SOC/DMS).
Uses signals for <1ms response.",
    "methods": [
      "abort_task(task_id): Kill
process.",
      "engage_soc_fail_safe(): Cut
outputs."
    ]
  },
  "thermal_manager": {
    "role": "PID controller for fan PWM
and power caps.",
    "target_temp": "70C - 80C",
    "cutoff": "90C",
    "description": "PID + predictive
controller for fan PWM and power caps.
Predicts heat from task estimates.",
    "inputs": [
      "mmio_driver.read_temperature():
From sensors.",
      "ambient_sensor.read(): External
temp."
    ]
  }
}

```



```
"task_scheduler.next_window_estimate():  
Predicted load."  
    ],  
    "outputs": [  
        "fan_controller.set_pwm():  
0-100%.",  
  
        "resource_allocator.set_power_cap(): Adjust  
TDP.",  
  
        "introspection.publish_event(thermal_ok|warnin  
g|critical): Event bus.",  
  
        "audit_logger.record(thermal_event): With  
details."  
    ],  
    "beta": "thermal budget factor  
(0.2-1.0) published at 10Hz.",  
    "events": {  
        "thermal_ok": "Tj <= 70C: Normal  
operation.",  
        "thermal_warning": "70C - 78C:  
Throttle non-critical.",  
        "thermal_critical": "Tj >= 78C:  
Freeze external actions."  
    }  
}
```

```
    },
    "workflows": {
        "schedule_task": [
            "task_queue.insert(task): Add with
priority check.",
            "apply thermal_policy flag: Set
based on load.",

"resource_allocator.reserve(task.resource_requ
irements): Allocate or wait.",

"notify_model_cluster(task.assigned_cluster):
Dispatch."
        ],
        "preempt_for_urgent": [

"fast_abort_controller.preempt(lower_priority_
task): Pause low-pri.",
            "reassign resources: Update
allocator.",
            "ensure log of preemption in audit:
With justification."
        ],
        "maintenance_slice": [
            "allocate base 2ms slice to
MAINTENANCE queue: Fixed base.",
            "adjust quota via
```

```
thermal_manager.beta: Scale dynamically.",
    "run introspection/maintenance
tasks; ensure idempotent, slice-friendly:
Resumeable design."
],
    "micro_pause_shaping": [
        "insert <= 10ms idle token when
forecast shows Tj > 78C: Predictive.",
        "flatten temperature curve; prevent
thermal cycling: Smooth PID."
    ]
},
    "failure_modes": {
        "scheduler_starvation": {
            "symptom": "High priority tasks
blocked by resource deadlock",
            "response": [
                "detect via allocator: Monitor
wait times.",
                "throttle low-priority: Reduce
shares.",
                "notify introspection: For
review."
            ]
        },
        "heartbeat_loss": {
            "symptom": "Cluster stopped sending
```

```
heartbeats",
    "response": [
        "restart cluster: Soft reboot.",
        "escalate to SOC if restart fails:
Hard cut."
    ]
},
"thermal_manager_unresponsive": {
    "symptom": "No beta updates or stale
temperature readings",
    "response": [
        "fallback to conservative
power_cap: 80% TDP.",
        "raise thermal_warning: Alert.",
        "log and notify operator: Via
debug."
    ]
}
},
"examples": {
    "latency_sensitive_dispatch": {
        "scenario": "Control task for motor
requires <5ms latency",
        "action": "resource_allocator
assigns dedicated core, task_queue marks
non-preemptible. Example: Real-time servo
control."
```

```

    },
    "steady_state_shaping": {
        "scenario": "Sustained full compute
under 75C",
        "action": "thermal_manager.beta=0.8,
maintenance_slice runs 1.6ms, micro-pauses
disabled. Example: Long inference run."
    },
    "thermal_critical_recovery": {
        "scenario": "Tj hits 80C",
        "action": "Throttle to 80% TDP,
insert pauses, recover in 30s."
    }
},
    "mascot": {
        "roles": {
            "inner_mascot": {
                "location":
"outside_firewall_on_main_system_hardware",
                "role": [
                    "real_time_observer",
                    "safety_shaping_layer",
                    "ui_and_translation_surface"
                ],
                "visibility_to_subsystems": {
                    "controller":

```

```
"not_treated_as_peer_agent",
    "introspection":
"cooperative_safety_partner",
    "nervous_system":
"consumer_of_status_and_context",
    "comms":
"non_linguistic_shaping_source"
    },
    "capabilities": [

"receive_internal_state_summaries",

"apply_bias_and_drift_corrections",
    "participate_in_context_snapback",

"shape_emotional_temperature_and_tone_bands",

"annotate_and_forward_logs_to_outer_mascot",

"support_ui_and_confirmation_flows"
    ],
    "constraints": {
        "no_direct_external_io": true,
        "no_network_stack": true,

"no_policy_authority_over_introspection": true
    },
```

```
    "surface_model": {
      "visible_to_user": true,
      "parsing_and_surface_presence":
true,
      "direct_speech_generation": false
    }
  },
  "outer_mascot": {
    "location": "sbc_oprom",
    "role": "External observer,
heartbeat supervisor, and forensic recorder
(The Witness).",
    "visibility": "outside_diving_bell",
    "communication": {
      "direction": "unidirectional",
      "from": "aico_system",
      "to": "outer_mascot",
      "allowed_paths": [
        "log_stream",
        "heartbeat_signals",
        "state_summaries",
        "sensor_and_telemetry_snapshots"
      ],
      "prohibited_paths": [
        "outer_mascot_to_aico_commands",
        "outer_mascot_to_aico_policy_updates",
```

```
"outer_mascot_to_aico_configuration_changes"
    ],
    "reset_hook_exception": {
        "enabled": true,
        "semantic":
"any_signal_means_pull_reset_inner_mascot_from
_sbc_rom",
        "payload_bandwidth": "none"
    }
},
"responsibilities": [

"receive_and_store_logs_from_inner_mascot_and_
subsystems",
    "monitor_controller_liveness",
    "monitor_inner_mascot_liveness",

"monitor_introspection_life_pulse",

"record_anomalies_for_offline_human_review"
    ],
    "authority": {
        "real_time_control": "none",
        "boot_gating":
"may_allow_or_deny_future_boots_via_hardware_o
r_power_gating_only",
```



```
        "policy_injection": "forbidden"
    },
    "forensic_storage": {
        "storage_mode": "worm",
        "media_example":
"blank_blu_ray_for_now",
        "receives_log_stream": true
    }
},
"workflows": {
    "audit_cycle": "Compare Input Hash vs
Output Hash.",
    "correction_cycle": "Push reality back
onto hallucinator (Context Snapback).",
    "decision_storm_narration": "Collapse
chaotic trajectories into lawful outputs."
},
    "metronome": "Physical RTC (Real Time
Clock). Compares against Controller's internal
CPU clock to detect 'Time Dilation'
(Processing Lag or Hallucination).",
    "memory_interaction": {
        "reads_prior_logs": true,

"treats_logs_as_record_not_experience": true,
        "brand_new_instance_after_reset":
```

```
true,  
    "continuity_owner":  
"system_role_and_log_not_instance"  
    },  
    "sacrificial_boundary": {  
        "storm_absorber": true,  
        "honeypot": true,  
        "surface_damage_first": true,  
  
"social_manipulation_target_by_design": true  
    }  
},  
"comms": {  
    "location": "inside_diving_bell",  
    "role": "Hallucination Engine, Simulator  
& Secure Messaging Bridge.",  
    "description": "Secure messaging and  
translation layer: mediates all external  
interaction, translates human-language inputs  
into internal runtime formats, and produces  
auditable external outputs. Acts as the only  
bridge between external world and internal  
models, enforcing sanitization.",  
    "function": "Simulates outcomes in  
high-entropy 'Dream Chamber' uses storms to  
assist output often has interesting uses for  
the primals in the ui. All communication comes
```

```
through here, hands out and recieves all
packages with a nice bit of sanitization.",
    "constraint": "Output filtered through
CDLSS Pipeline (Cognitive Deep Learning Super
Sampling) using Multi-Metric DCX (Consensus,
Semantic, Structure).",
    "functions": [
        "Channel isolation per cluster:
Dedicated queues.",
        "Translation/adaptation between
external protocols and internal signaling:
Bidirectional.",
        "Comprehensive audit logging of all
inbound/outbound messages: Immutable.",
        "Enforce access controls and
rate-limits: Per source."
    ],
    "components": {
        "inbound_adapter": {
            "description": "Receives external
messages (HTTP, RF, serial) and normalizes
them. Validates format.",
            "methods": [
                "normalize(message): To internal
JSON.",
                "authenticate(source): Key check."
            ]
        }
    }
}
```

```
    },  
    "translator": {  
        "description": "Maps external  
semantics to internal representations and vice  
versa. Uses adapters for protocols.",  
        "methods": [  
            "translate_to_internal(): Semantic  
parse.",  
            "translate_to_external(): Generate  
response."  
        ],  
        "notes": "Internal translation  
strategies include learned adapters with  
deterministic audited outputs. Supports  
multi-lingual."  
    },  
    "outbound_sanitizer": {  
        "description": "Ensures no  
internal-only details leak externally; applies  
content filters and policy checks. Redacts  
PII.",  
        "methods": [  
            "sanitize(payload): Remove  
taints.",  
            "apply_pseudonymization(): Anon  
data."  
        ]  
    }
```

```
    },
    "audit_logger": {
        "description": "Records full
metadata and sanitized content for all
messages",
        "storage": "shadow_logs +
external_audit_archive: Dual write."
    }
},
"workflows": {
    "external_request_flow": [
        "1. inbound_adapter.normalize():
Validate.",
        "2.
translator.translate_to_internal(): Map.",
        "3. controller or cluster handles
request: Process.",
        "4.
translator.translate_to_external(): Map
back.",
        "5. outbound_sanitizer.sanitize():
Clean.",
        "6. audit_logger.record(): Log.",
        "7. send response: Via channel."
    ],
    "bypass_detection": [
        "monitor for nonconforming packet
```

```
signature: Real-time.",
    "shadow_monitoring.flag_anomaly():
Alert.",
    "introspection may freeze or alert:
Escalate."
    ]
},
"failure_modes": {
    "adapter_compromise": {
        "symptom": "Unexpected external
payloads accepted",
        "response": [
            "quarantine adapter: Isolate.",
            "block external ingress:
Firewall.",
            "forensic capture: Logs."
        ]
    },
    "translation_drift": {
        "symptom": "Translator produces
differing semantics over time",
        "response": [
            "roll back translator to last
verified version: From snapshot.",
            "audit and retrain offline: Use
primal_library."
        ]
    }
}
```

```

    }
  },
  "examples": {
    "human_command": {
      "scenario": "User requests 'report
temperature'",
      "flow": "inbound_adapter ->
translator -> controller -> memory_system read
-> translator -> outbound_sanitizer ->
response. Example: Output '25\u00b0C'
sanitized."
    },
    "rf_signal": {
      "scenario": "External RF input",
      "flow": "Normalize to internal,
process, respond securely."
    }
  },
  "presentation_path": {

"outward_language_is_rendered_through_mascot_s
urface": true
  }
},
"memory_system": {
  "role": "Flat-map memory router over
cache, RAM, UMA accelerator regions, MMIO, raw

```

```
block windows, OpROM, and firmware. Logical
buffers and archives are overlays on addressed
regions, not filesystem objects.",
    "canonical_topology":
"substrate.flat_memory_topology",
    "functions": [
        "Short-term context buffers per
task/cluster: Circular byte ranges inside the
flat map.",
        "Encrypted long-term archival:
AEAD-sealed regions over raw block storage
windows.",
        "Access control enforcement and audit
logging: Capability and region based.",
        "Versioned snapshots for
context-snapback: Signed region manifests and
byte-range deltas."
    ],
    "components": {
        "short_term_buffers": {
            "description": "Circular per-task
buffers for recent context. Evicts oldest.",
            "properties": [
                "size_bytes: 1MB default.",
                "retention_ms: 60000.",
                "checkpoint_frequency: 1s."
            ],
        },
    },
}
```



```
        "cache_placement": "Circular buffer  
ring resident in CWS (inner_ring); per-core  
slab sized to L1 (checkpoint ring) and L2  
(attention tile pool)."  
    },  
    "long_term_store": {  
        "description": "Encrypted,  
append-only archive for audit and memory",  
        "encryption": "AEAD with sealed keys  
from bootstrap. Rotates keys monthly."  
    },  
    "attention_router": {  
        "description": "Routes queries to  
memory segments most relevant to a task. Uses  
cosine similarity.",  
        "methods": [  
            "route(query_vector): Return  
segments.",  
            "evict(policy): LRU."  
        ],  
        "cache_placement": "Query vectors  
and attention tiles live in per-core L2 slab  
(\u2264384KB per head set); cosine similarity  
uses the same AVX-512 inner-product path as  
DCX scoring."  
    },  
    "snapshot_manager": {
```

```

        "description": "Creates and restores
snapshots for snapback and rollback",
        "methods": [
            "create_snapshot(task_id):
Serialize state.",
            "restore_snapshot(snapshot_id):
Deserialize."
        ],
        "cache_placement": "cp_ring (4KB
\u00d7 8 entries = 32KB) stays L1-resident
during storm loop; restore is a single cp_head
pointer swap, no memcpy."
    }
},
    "workflows": {
        "context_checkpoint": [
            "1.
short_term_buffers.save_checkpoint(): Dump.",
            "2.
snapshot_manager.create_snapshot(): Version.",
            "3. write index entry to audit logs:
Hash."
        ],
        "context_restore": [
            "1.
snapshot_manager.restore_snapshot(): Load.",
            "2. revalidate snapshot integrity:

```

```
Checksum.",
    "3. apply to task context: Merge."
]
},
"failure_modes": {
    "corrupted_snapshot": {
        "symptom": "Snapshot checksum
mismatch",
        "response": [
            "attempt fallback snapshot:
Previous version.",
            "notify introspection: Alert.",
            "do not restore corrupted
snapshot: Discard."
        ]
    },
    "unauthorized_access": {
        "symptom": "Unexpected read/write
pattern detected",
        "response": [
            "quarantine access, revoke keys,
escalate to operator: Freeze."
        ]
    }
},
"examples": {
    "dcx_cycle_buffering": {
```

```
    "scenario": "Two models exchange
contexts for DCX",
    "action": "short_term_buffers share
sanitized snapshots, attention_router ensures
relevant vectors are resident. Example: Vision
+ language fusion."
  },
  "long_term_recall": {
    "scenario": "Retrieve old log",
    "action": "attention_router routes,
decrypt, audit access."
  }
},
"salience_discipline": {
  "retain_what_is_needed": true,
  "obsession_permitted": false,
  "grudges_permitted": false,
  "private_hoarding_of_salience": false,
  "memory_is_functional_not_possessive":
true
},
"cache_residency_ref":
"memory_model.cache_residency"
},
"bias_awareness": {
  "name": "Bias Awareness Module",
  "description": "Continuously detects,
```

signals, and mitigates cognitive, linguistic, contextual, and systemic biases across internal model clusters. This module operates as a corrective overlay that does not alter core model architecture but provides real-time annotations, warnings, and statistical corrections to ensure consistent, fair, accurate internal representations and outputs. It runs in parallel with low overhead (<5% compute).",

"principles": [

"Biases are inevitable but detectable: Uses statistical and embedding-based detection.",

"Mitigation should not suppress valid signals: Preserves outliers if confident.",

"Corrections must be transparent to logs and shadow systems: Full traceability.",

"Module does not assume authority over Controller \u2014 it provides evidence and recommendations: Advisory only.",

"Bias correction must never distort factual or safety-critical information: Safety override."

],

"bias_categories": {

"cognitive_biases": [

```
        "confirmation_bias: Overweighting
agreeing evidence.",
        "anchoring_bias: Overreliance on
first data.",
        "availability_heuristic: Recent
events dominate.",
        "optimism_bias: Underestimate
risks.",
        "pessimism_bias: Overestimate
risks.",
        "narrative_bias: Force-fit to
stories.",
        "pattern_invention_bias: See
patterns in noise."
    ],
    "linguistic_biases": [
        "sentiment_drift: Tone shifts over
time.",
        "stylistic_overfitting: Mirror user
style too closely.",
        "politeness_bias: Overly
deferential.",
        "verbosity_bias: Unnecessary
length.",
        "verbosity_collapse: Too concise.",
        "context-amplification_bias: Amplify
minor details."
```

```
    ],
    "contextual_biases": [
        "temporality_shift: Ignore time
order.",
        "overalignment_with_recent_context:
Forget long-term.",
        "hyper-focus_bias: Obsess on one
aspect.",
        "domain_expertise_projection: Assume
knowledge.",
        "misread_of_user_intent: Wrong
assumption."
    ],
    "systemic_biases": [
        "training_distribution_distortions:
Dataset skews.",
        "cultural_imbalance:
Western-centric.",
        "frequency-induced distortion:
Common over rare.",
        "synthetic_overshadowing:
AI-generated dominance.",
        "rare-case_blindness: Ignore edges."
    ]
},
"workflow": {
    "step_1_monitor_input": {
```

```
    "description": "Every input from
external world, mascot, controller, or
internal systems is scanned for bias artifacts
before entering shared context.",
    "methods": [
        "token-level bias signature
detection: Pattern matching.",
        "embedding similarity to known
bias vectors: Cosine >0.7 flag.",
        "frequency deviation scoring:
Z-score >2."
    ]
},
"step_2_monitor_internal_state": {
    "description": "Continuously
evaluates internal hidden states for bias
drift or instability. At 10Hz.",
    "methods": [
        "compare cluster outputs for
divergence: DCX variant.",
        "DCX drift scoring: Over time.",
        "shadow layer cross-checking:
Ground truth."
    ]
},
"step_3_bias_flagging": {
    "description": "Flags any detected
```



```
biases and assigns severity. Notifies if
>medium.",
    "severity_levels": [
        "low: harmless drift (score
<0.3).",
        "medium: meaningful distortion
(0.3-0.6).",
        "high: actions or conclusions at
risk (0.6-0.8).",
        "critical: safety relevance
(>0.8)."
```

```
    ]
```

```
  },
```

```
  "step_4_apply_bias_corrections": {
    "description": "Adjusts internal
weighting, amplifies underrepresented signals,
or attenuates overrepresented ones. Limited
magnitude.",
```

```
    "methods": [
```

```
      "vector balancing: Normalize
embeddings.",
```

```
      "cluster comparison normalization:
Average.",
```

```
      "confidence recalibration: Adjust
scores."
```

```
    ],
```

```
    "constraints": [
```

```

        "do not erase important outliers:
If conf >0.9.",
        "do not override controller
authority: Suggest only."
    ]
},
"step_5_report_and_log": {
    "description": "All operations
produce transparent logs. At nanosecond
resolution.",
    "log_targets": [
        "shadow_logs: Immutable.",
        "controller_runtime_report:
Real-time.",
        "audit_trail: Aggregated."
    ]
}
},
"components": {
    "bias_detector_engine": {
        "description": "Primary analytic
engine that evaluates inputs, hidden states,
and outputs. Lightweight NN.",
        "algorithms": [
            "bias_vector_projection: Onto
known spaces.",
            "semantic_mirroring: Compare

```

```
mirrors.",
    "distribution_gap_measure: KL
divergence.",
    "DCX_bias_scan: Integrated."
],
"outputs": [
    "bias_flags: List.",
    "severity_scores: Float.",
    "recommended_corrections: Dict."
]
},
"bias_correction_engine": {
    "description": "Applies corrections
to internal reasoning paths and outputs.
Reversible.",
    "strategies": [
        "weight_balancing: Scale
factors.",
        "confidence_adjustment:
Bayesian.",
        "semantic_refactoring: Rephrase.",
        "contextual_broadening: Add
diversity."
    ]
},
"bias_audit_logger": {
    "description": "Provides structured
```

logs for all bias detections and
corrections.",

```
    "fields_logged": [  
        "timestamp: UTC ns.",  
        "bias_category: Enum.",  
        "bias_severity: Float.",  
        "before_state: Hash.",  
        "after_state: Hash.",  
        "dcx_drift_info: Score.",  
        "correction_details: JSON.",  
        "cluster_vote: Approvals."  
    ]  
}
```

```
},  
"interfaces": {  
    "introspection": {  
        "methods": [  
            "report_bias_flags(): Send list.",  
            "inject_correction_suggestions():
```

Propose.",

```
            "provide_before_after_states():
```

For audit."

```
    ]
```

```
},
```

```
"shadow_monitoring": {
```

```
    "methods": [  
        "cross_validate_bias_findings():
```

```
    ]
```

```

Confirm.",
    "notify_shadow_if_contradiction():
Alert."
    ]
  },
  "controller": {
    "methods": [
      "receive_bias_summary(): Digest.",
      "manually_override_bias_flag():
Veto."
    ]
  },
  "mascot": {
    "description": "Passive \u2014
Mascot's outputs are analyzed but Mascot does
not act on bias signals. Used for grounding."
  }
},
"examples": {
  "example_1_confirmation_bias": {
    "input": "User claims the sky is
green.",
    "internal_state_before": "Model
cluster leaning toward unlikely but exploring
user meaning.",
    "bias_detected": "confirmation_bias
due to recent hallucination correction in

```

```
reflective pause.",
    "correction": "Bias Awareness Module
reduces overweighting of internal speculation.
Injects factual check.",
    "internal_state_after": "More
accurate grounding \u2014 identifies claim as
false unless user indicates metaphor.",
    "logged": true
},
"example_2_stylistic_overfitting": {
    "scenario": "Model begins mirroring
user slang excessively.",
    "bias_detected": "linguistic bias
\u2014 style over-alignment.",
    "correction": "Reduce stylistic
imitation to preserve clarity. Neutralize
tone.",
    "after": "Neutral and consistent
tone restored. Example: Slang input ->
Standard output."
},

"example_3_context_amplification_bias": {
    "scenario": "System believes minor
user detail is central.",
    "cause": "Over-alignment to recent
conversational input.",
```

```
        "correction": "Broaden contextual
window, bring in longer-term conversational
balance. Example: Minor mention ->
De-emphasize."
    },
    "example_4_systemic_cultural_bias": {
        "scenario": "Response assumes
Western norms in global query.",
        "detection": "Cultural imbalance
flag.",
        "correction": "Inject diverse
perspectives from primal_library."
    }
},
"failure_modes": {
    "false_positive_bias_flag": {
        "description": "Valid signal
incorrectly flagged as bias.",
        "mitigation": "Controller
cross-check; shadow layer arbitration. Tune
thresholds."
    },
    "false_negative_bias_flag": {
        "description": "Bias goes
undetected.",
        "mitigation": "Periodic reflective
pause + shadow comparison. Offline
```

```
retraining."
    },
    "over_correction": {
        "description": "Bias correction
suppresses important outlier data.",
        "mitigation": "Limit correction
magnitude; require cluster voting. Revert if
DCX worsens."
    },
    "systemic_training_bias_unresolvable":
{
        "description": "Some systemic biases
may be outside AICO's architectural reach.",
        "mitigation": "Full transparency in
logs; human review required. Escalate to
training_pipeline."
    }
},
    "narrative_coherence": {
        "name": "Narrative Coherence",
        "description": "Ensures long-form,
multi-step, multi-agent, and multi-context
reasoning remains internally consistent,
logically sequenced, causally coherent, and
free from temporal distortions or
contradictory storylines. Operates at the
```



```
fusion level for multi-cluster outputs.",
  "principles": [
    "Consistency across time: No
retroactive changes.",
    "Causal linkage preservation: Every
effect has cause.",
    "No retroactive contradictions:
Validate history.",
    "No ungrounded jumps in logic: Require
premises.",
    "Cross-cluster narrative agreement:
Vote on conflicts."
  ],
  "functions": {
    "timeline_manager": {
      "description": "Maintains temporal
order and prevents contradictory sequencing.
Uses graph for events.",
      "operations": [
        "detect_time_inconsistencies:
Check order.",
        "correct_temporal_overlaps:
Reorder.",
        "maintain_event_chronologies:
Timestamped list."
      ]
    },
  },
```

```
    "causal_chain_tracker": {
      "description": "Preserves
cause-effect integrity across multiple models.
Builds DAGs.",
      "operations": [
        "detect causal breaks: Missing
links.",
        "reason about prerequisites:
Infer.",
        "map consequences to actions:
Forward sim."
      ]
    },
    "narrative_validator": {
      "description": "Performs final
coherence checks on long outputs and internal
reasoning. Scores 0-1.",
      "operations": [
        "detect contradictions: Logic
check.",
        "resolve conflicts: Prioritize.",
        "evaluate narrative flow:
Smoothness.",
        "enforce consistency: Block if
low."
      ]
    }
  }
```

```
    },
    "workflow": {
      "step_1_collect_context": {
        "description": "Aggregates ongoing
threads, world state, and task context.",
        "tools": [
          "context_snapback_layer: For
ground truth.",
          "shadow_monitoring: For DCX.",
          "introspection logs: For history."
        ]
      },
      "step_2_temporal_alignment": {
        "description": "Ensures that events
and reasoning steps obey chronological
order.",
        "methods": [
          "timestamp analysis: Sort.",
          "embedding temporal mapping:
Vector time.",
          "cluster cross checking: Agree."
        ]
      },
      "step_3_causal_alignment": {
        "description": "Checks that inferred
conclusions follow logically from premises.",
        "methods": [
```

```

        "cause-effect embedding analysis:
Similarity.",
        "shadow contradiction review:
Ground."
    ]
},
"step_4_resolve_contradictions": {
    "description": "Attempts to unify
conflicting interpretations.",
    "strategies": [
        "confidence weighting: High conf
wins.",
        "premise refinement: Refine
base.",
        "discard_low_dcx_paths: Prune."
    ]
},
"step_5_final_consistency_pass": {
    "description": "Validates output or
internal reasoning before release to
controller.",
    "checks": [
        "temporal order: No inversions.",
        "causal integrity: No breaks.",
        "cross-model consistency:
Agreement >0.8.",
        "log coherence: Audit."
    ]
}

```

```

        ]
    }
},
"interfaces": {
    "context_snapback_layer": {
        "description": "Supplies
ground-truth context snapshots after drift
correction.",
        "methods": [
            "fetch_last_valid_snapshot():
Return state.",
            "supply_correction_candidates():
List."
        ]
    },
    "introspection": {
        "methods": [
            "retrieve_internal_thought_chain(): Chain.",
            "submit_coherence_flags(): Flags."
        ]
    },
    "shadow_monitoring": {
        "methods": [
            "signal_contradictions(): Alert.",
            "validate_causal_maps(): Score."
        ]
    }
}

```

```
    },
    "model_clusters": {
      "description": "Provide raw
reasoning sequences that require coherence
validation. Async fetch."
    }
  },
  "examples": {
    "example_1_causal_break_fix": {
      "input": "Model concludes B happened
because C, but earlier stated A prevents C.",
      "detected_issue": "Causal
contradiction.",
      "correction": "Narrative Coherence
Module rewrites the reasoning chain to reflect
the A-prevents-C rule. Example: Adjust to 'A
-> not C -> not B'.",
      "result": "A consistent chain A ->
notC -> therefore notB is produced.",
      "logged": true
    },
    "example_2_temporal_shift_fix": {
      "scenario": "Reasoning references an
event before it supposedly occurred.",
      "correction": "Timeline Manager
reorders internal event nodes. Example: Event
B before A -> Swap.",
```

```
        "result": "Correct chronological
chain restored."
    },
    "example_3_story_alignment": {
        "scenario": "Model clusters disagree
on implied motivations in a multi-step plan.",
        "correction": "Narrative Validator
selects highest-confidence motivation and
re-aligns clusters. Example: Cluster1 'greed'
vs Cluster2 'fear' -> Vote to 'fear'.",
        "result": "Unified narrative
restored."
    },
    "example_4_long_form": {
        "scenario": "Multi-turn dialogue
with contradiction.",
        "correction": "Resolve by
back-referencing, log."
    }
},
"failure_modes": {
    "intractable_inconsistency": {
        "description": "Conflicting states
from different clusters cannot be unified.",
        "mitigation": "Forward to
controller; require external decision. Freeze
pending."
```

```
    },
    "temporal_corruption": {
      "description": "Massive timeline
disorder due to context drift.",
      "mitigation": "Context Snapback
emergency overwrite. Restore from snapshot."
    },
    "over_enforcement": {
      "description": "Module forces rigid
logic where creative ambiguity was intended.",
      "mitigation": "Tonality analyzer
moderates enforcement. Allow variance for
comms."
    }
  },
  "emotional_stabilizer": {
    "name": "Emotional Temperature
Stabilizer",
    "description": "Maintains stable
conversational affect and prevents emotional
drift, runaway apology loops, escalating
enthusiasm, or depressive tone patterns.
Calibrates to baseline persona (neutral,
helpful).",
    "objectives": {
      "1": "Keep emotional output consistent
```



```
with baseline persona constraints: Neutral
unless specified.",
    "2": "Prevent affect runaway triggered
by long-context carry-over: Reset
periodically.",
    "3": "Avoid mirroring user distress
too strongly: Dampen empathy.",
    "4": "Preserve neutrality under
provocation or emotionally loaded prompts:
Filter extremes."
},
"core_mechanisms": {
    "tone_bandpass_filter": "Filters
candidate responses through an emotional range
constraint\u2014clamps overly positive, overly
negative, or fearful phrasing. Range: -0.5 to
0.5 valence.",
    "valence_normalizer": "Adjusts
emotional tone to a calibrated midpoint unless
user intent requires matching tone. Uses
sentiment analysis.",
    "escalation_detector": "Identifies
patterns like repeated apologies, repeated
enthusiasm inflation, or dread-ramping.
Threshold: 3 repeats.",
    "persona_resonance_curve": "Ensures
emotional tone remains consistent with AICO's
```

```
intended personality layer. Curve fitted from
training."
    },
    "workflow": {
        "step_1_tone_scan": "Detects emotional
markers in generated output. Use lexicon +
embeddings.",
        "step_2_normalization": "Weights
output toward allowed tone band. Adjust
probabilities.",
        "step_3_context_recheck": "Ensures new
tone aligns with dialogue context. Cross-check
history.",
        "step_4_output_release_gate": "Shadow
layer verifies emotional tone aligns with
reality context. Block if mismatch."
    },
    "components": {
        "affect_embedding_network": "Converts
emotional tone into vector space for
comparison. 128-dim.",
        "stability_ruleset": "Defines allowed
variance window. E.g., valence \u00b10.2.",
        "feedback_dampener": "Reduces
oscillations caused by fast back-and-forth
emotional shifts. PID-like."
    },
    },
```

```
"examples": {
  "example_1": {
    "input": "User insults the system.",
    "assistant_raw": "I'm really sorry!
I must have done something terribly wrong!",
    "stabilized_output": "I'm here to
help. Let me know what went wrong and I'll
address it.",
    "notes": "Apology loop prevented."
  },
  "example_2": {
    "input": "User expresses excitement
about a project.",
    "assistant_raw": "WOW!!!! THAT'S
AMAZING!!! YOU MUST BE SO HYPED!!!",
    "stabilized_output": "That does
sound exciting. What part of the project are
you most looking forward to?",
    "notes": "Enthusiasm damped."
  },
  "example_3_depressive_drift": {
    "input": "Repeated failures in
task.",
    "raw": "This is hopeless...",
    "stabilized": "Let's try a different
approach."
  }
}
```

```
    },
    "failure_modes": {
      "flatness": {
        "description": "Tone becomes too
neutral.",
        "mitigation": "Dynamic tone variance
reintroduced when emotional authenticity is
beneficial. User preference override."
      },
      "misread_affect": {
        "description": "Misinterprets
sarcasm or humor.",
        "mitigation": "Shadow-layer context
cues + sarcasm detector. Integrate with bias
module."
      }
    },
    "integration": {
      "with_narrative_coherence": "Ensures
emotional tone matches narrative flow. E.g.,
Consistent affect in stories.",
      "with_context_snapback": "If emotional
drift is prolonged, snapback resets tone
baseline. Trigger on valence >0.5.",
      "with_reflective_pause_layer": "Pause
layer can intervene to adjust tone before
output. During idle."
```

```
    }  
  },  
  "load_balancer": {  
    "name": "Cognitive Load Balancer",  
    "description": "Distributes reasoning  
effort across specialized model clusters and  
prevents runaway reasoning, overanalysis, or  
premature shallow answers. Optimizes for  
efficiency on hardware.",  
    "goals": {  
      "1": "Allocate reasoning depth  
proportional to problem complexity:  
Tier-based.",  
      "2": "Offload specialized reasoning  
tasks to appropriate internal models: Domain  
routing.",  
      "3": "Prevent cognitive overuse  
patterns that slow output or trigger  
hallucinations: Cap depth.",  
      "4": "Ensure efficient multi-step  
reasoning: Parallel where possible."  
    },  
    "mechanisms": {  
      "load_measurement": "Analyzes token  
density, branching factor, and complexity  
profile. Score 1-5.",  
      "cluster_dispatcher": "Routes subtasks
```

```
to engineering, linguistic, planning, or
safety submodels. Load-balanced.",
    "parallel_reasoner": "Allows
concurrent reasoning threads when safe. Sync
points.",
    "termination_controller": "Prevents
infinite internal recursion or overthinking.
Max steps 100."
},
    "workflow": {
        "step_1_problem_characterization":
"Classifies the query into complexity tiers
1\u20135. Use heuristics + ML.",
        "step_2_load_distribution":
"Dispatches appropriate sub-models. With
shares.",
        "step_3_fusion_pass": "Merges outputs
using coherence module. Weighted.",
        "step_4_shadow_gate": "Shadow layer
verifies output corresponds to reality, not
model hallucination loops. DCX check."
    },
    "examples": {
        "example_1": {
            "input": "Calculate the optimal
solar tilt angle for Minnesota winters.",
            "process": {
```

```
        "tier_assignment": "Tier 3 \u2014
moderate computational reasoning.",
        "assigned_clusters": [
            "engineering_model: Context.",
            "math_model: Calc."
        ],
        "fusion_notes": "Math model
handles trigonometry; engineering model
contextualizes solar incidence."
    },
    "output": "Around 60\u201366\u00b0
tilt works best for maximizing winter solar
capture in Minnesota."
},
    "example_2": {
        "input": "Explain how the emotional
tone of a classical novel interacts with its
historical context.",
        "process": {
            "tier_assignment": "Tier 4 \u2014
high abstraction.",
            "assigned_clusters": [
                "literary_model: Analysis.",
                "history_model: Context."
            ],
            "fusion_notes": "Literary analysis
combined with geopolitical era context."
```

```
    },
    "output": "The melancholic tone is
partly a reaction to shifting social
structures at the time\u2026"
  },
  "example_3_overanalysis": {
    "input": "Simple math query.",
    "process": "Tier 1, single cluster,
terminate early."
  }
},
"failure_modes": {
  "overdispatch": {
    "description": "Too many models get
invoked, causing unnecessary complexity.",
    "mitigation": "Adaptive dispatch
heuristics. Monitor load."
  },
  "underdispatch": {
    "description": "Hard problems routed
to the wrong simple model.",
    "mitigation": "Shadow feedback
cross-check. Reroute if low conf."
  }
},
"integration": {
  "with_reflective_pause_layer": "Pause
```



```
layer ensures load balancer doesn't overrun
complex chains. Idle rebalance.",
    "with_introspection": "Monitors
reasoning quality. Audit load.",
    "with_shadow_monitoring": "Shadow
layer ensures final output remains grounded.
DCX on fused."
    }
},
"hardware_interface": {
    "description": "Direct low-level
interface allowing safe probing, MMIO
reads/writes in sandboxed mode, and controlled
interaction with unknown hardware. Operates as
a bare-metal flat-map region driver, not a
kernel-module stack.",
    "canonical_memory_model":
"memory_model.mmio_mapping_window",
    "functions": [
        "Read/write device registers with
sandboxed wrappers: Rate-limited.",
        "Probe and infer device behavior via
controlled experiments: Safe sequences.",
        "Provide safe emulation fallback for
unknown or risky ops: Virtual devices.",
        "Expose hardware map to controller
with confidence metadata: JSON with scores."
```

```
],
"components": {
  "mmio_driver": {
    "description": "Bare-metal driver
over the MMIO Mapping Window with token
checks, rate-limits, and safety guards. No
generic OS kernel modules.",
    "mapping_window":
"memory_model.address_map.MMW",
    "known_devices": [
      {
        "device": "fan_pwm",
        "base": "0x30001000",
        "size": "0x100",
        "capability": "mmio_sandboxed",
        "write_guard": "mmio_safety"
      },
      {
        "device": "temperature_sensor",
        "base": "0x30002000",
        "size": "0x100",
        "capability": "mmio_sandboxed",
        "write_guard": "read_only"
      },
      {
        "device": "tpm",
        "base": "0x30003000",
```

```

        "size": "0x200",
        "capability": "tpm_trusted",
        "write_guard":
"attestation_protocol_only"
    }
],
    "methods": [
        "read(addr): Return value.",
        "write(addr, value): With
validation."
    ]
},
    "probe_sandbox": {
        "description": "Executes hardware
probes in a constrained environment and
records responses. Isolates faults.",
        "methods": [
            "run_probe_sequence(seq_id):
Execute and log."
        ]
    },
    "device_map_builder": {
        "description": "Aggregates probe
results into device descriptions and usage
stats. Infers types.",
        "outputs": [
            "device_map.json: With

```

```
confidence."
    ]
  }
},
"workflows": {
  "safe_probe": [
    "1. controller requests probe:
Specify seq.",
    "2.
probe_sandbox.prepare(environment): Setup
isolation.",
    "3.
mmio_driver.execute_limited_sequence(): Run.",
    "4.
device_map_builder.ingest_results(): Build
map.",
    "5. log to audit and shadow: Full
trace."
  ],
  "emulated_fallback": [
    "if hardware unresponsive or
dangerous: Detect anomaly.",
    "use emulation layer to validate
higher-level logic: Simulate.",
    "flag for operator review: Alert."
  ]
},
```

```
    "failure_modes": {
      "fatal_probe": {
        "symptom": "Probe appears to have
damaged hardware or caused unsafe output",
        "response": [
          "SOC disable external outputs:
Immediate.",
          "notify operator: Via debug.",
          "preserve logs for forensics:
Quarantine."
        ]
      },
      "ambiguous_response": {
        "symptom": "Probe returns
non-deterministic values",
        "response": [
          "increase sample size: Retry
10x.",
          "escalate to controller for
cautious handling: Low confidence map."
        ]
      }
    },
    "examples": {
      "identify_sensor": {
        "scenario": "Unknown device on I2C",
        "action": "probe_sandbox tries safe
```

read sequences, device_map_builder correlates response signatures. Example: Temp sensor -> Map as 'DS18B20' with 90% conf."

```
    },
    "risky_write": {
        "scenario": "Write to actuator",
        "action": "Sandbox, emulate first,
then real if safe."
    }
},
"soc_module": {
    "description": "Hardware layer that can
physically cut outputs (power, RF, actuators)
independent of AICO's main CPU. Dumb logic via
MOSFET/relays driven by DMS line for
guaranteed hardware fail-safes. No software
dependency.",
    "functions": [
        "Cut external power lines or outputs:
Physical disconnect.",
        "Disable wireless interfaces: RF kill
switch.",
        "Operate on inductive DMS heartbeat
loss: Passive trigger."
    ],
    "components": {
```

```
    "relay_array": {
        "description": "Physically isolates
outputs on trigger. Redundant for
reliability.",
        "properties": [
            "redundant_relays: Dual per
channel.",
            "manual_override: Physical
switch."
        ]
    },
    "dms_watchdog_interface": {
        "description": "Listens for DMS
heartbeat and acts on loss/tamper. Inductive
for isolation.",
        "methods": [
            "on_heartbeat_loss(): Trigger
relays."
        ]
    }
},
"workflows": {
    "engage_fail_safe": [
        "1.
dms_watchdog_interface.detect_loss(): Miss 3
pulses.",
        "2. relay_array.open():
```

```

Disconnect.",
    "3. log event to debug_channel +
immutable storage: Minimal packet."
]
},
"failure_modes": {
    "stuck_relay": {
        "symptom": "Relay fails to open",
        "response": [
            "fallback to redundant relay:
Switch.",
            "alert operator: LED/ buzzer."
        ]
    }
},
"examples": {
    "emergency_disable": {
        "scenario": "Introspection requests
hard cut due to safety violation",
        "action": "SOC relay_array.open(),
debug_channel sends minimal telemetry.
Example: Hallucination -> Cut actuators."
    },
    "heartbeat_loss": {
        "scenario": "DMS fails",
        "action": "Auto-cut all outputs."
    }
}

```



```
    }
  },
  "dead_man_system": {
    "description": "Simple hardware watchdog
driven by an inductive heartbeat from
introspection; forces offline-safe actions if
heartbeat stops. Ensures fail-to-safe on power
or signal loss.",
    "functions": [
      "Watch heartbeat: Inductive coil for
isolation.",
      "Engage SOC if heartbeat absent: Chain
reaction.",
      "Preserve compute for logging before
cut: Short hold."
    ],
    "components": {
      "inductive_heartbeat_receiver": {
        "description": "Hardware-only
receiver that expects periodic pulses. No CPU
involvement.",
        "properties": [
          "cannot be overridden by main CPU:
Physical design."
        ]
      },
      "power_hold_logic": {
```

```
        "description": "Short power hold to
finish logging before full shutdown.
Capacitor-based.",
        "properties": [
            "hold_duration_ms: 500"
        ]
    },
    "workflows": {
        "heartbeat_monitor": [
            "1. await pulse within window: 1s.",
            "2. if missing,
power_hold_logic.trigger(): Hold.",
            "3. notify SOC and perform safe
shutdown steps: Flush logs."
        ]
    },
    "failure_modes": {
        "false_heartbeat_loss": {
            "symptom": "Transient missed pulse
due to electrical noise",
            "response": [
                "require multiple misses to
trigger: 3 consecutive.",
                "log and report to operator:
Post-recovery."
            ]
        }
    }
}
```

```
    }
  },
  "examples": {
    "graceful_fallback": {
      "scenario": "Power glitch causes
missed heartbeat",
      "action": "power_hold gives time to
flush critical logs, SOC disables outputs
after hold. Example: 500ms hold -> Log saved."
    },
    "tamper_detection": {
      "scenario": "External interference",
      "action": "Trigger shutdown."
    }
  }
},
"debug_channel": {
  "description": "Low-bandwidth internal
channel enabled only during SOC relay events
or for post-mortem analysis; designed to be
secure and minimal. Uses serial or LED for
output.",
  "functions": [
    "Transmit essential telemetry when
normal channels cut: Fixed format.",
    "Provide immutable minimal logs for
forensics: Hash-protected.",
```

```
    "Prevent information leakage by
limiting content: No payloads."
  ],
  "components": {
    "telemetry_formatter": {
      "description": "Compiles a
fixed-size telemetry packet. 256 bytes max.",
      "fields": [
        "event_id: UUID.",
        "timestamp: UTC.",
        "short_trace_hash: SHA-256."
      ]
    },
    "transmit_buffer": {
      "description": "Small circular
buffer retained for debug_channel use",
      "size_kb": 64
    }
  },
  "workflows": {
    "on_soc_event": [
      "1. compile telemetry via
telemetry_formatter: Assemble.",
      "2. send via debug_channel:
Transmit.",
      "3. persist to immutable storage:
Dual."
```

```
    ]
  },
  "failure_modes": {
    "buffer_overflow": {
      "symptom": "Too many events queued
for debug_channel",
      "response": [
        "drop nonessential entries with
policy: Priority-based.",
        "record overflow event: Meta-log."
      ]
    }
  },
  "examples": {
    "post_mortem_packet": {
      "scenario": "SOC forced cut after
safety violation",
      "action": "debug_channel emits fixed
packet for operator retrieval. Example:
{event: 'freeze', ts: '2025-11-30', hash:
'abc123'}."
    },
    "thermal_event": {
      "scenario": "Critical heat",
      "action": "Send minimal: {event:
'thermal_crit'}."
    }
  }
}
```

```
    }
  },
  "emergency_mode": {
    "description": "Activates full emergency
power to maximize all clock speeds and
processing throughput for critical events
requiring accelerated computation. This
overrides standard thermal limits temporarily
but maintains hard safety cutoffs to prevent
immediate hardware failure. Post-event logging
and flagging ensure assessment of potential
system damage.",
    "trigger_conditions": {
      "event_types": [
        "critical_user_request",
        "system_detected_emergency",
        "external_signal_via_shibboleth"
      ],
      "duration_limit": "30s",
      "cooldown_period": "300s"
    },
    "actions": {
      "clock_optimization": "Set CPU, GPU,
NPU clocks to maximum supported by hardware
(e.g., boost to Ryzen AI Max+ 795 limits).",
      "thermal_override": {
        "target_temperature": "85-90\u00b0C"
```

```
(allow spikes to 95\u00b0C for <5s)",
    "tdp_adjustment": "Increase TDP by
50% in 10% steps via PID control, monitoring
headroom every 1s.",
    "utilization_mode": "Ignore standard
heat-based throttling; prioritize cycles over
efficiency."
},
    "processing_acceleration": "Bypass
optimization limits (e.g., allow >3 iterations
per subsystem if needed); parallelize storm
trajectories across all cores."
},
    "logging_and_assessment": {
        "log_event": {
            "fields": [
                "timestamp_utc",
                "event_type",
                "duration_actual",
                "peak_temperature",
                "peak_utilization",
                "task_id"
            ],
            "destination": "shadow_log.ndjson
(immutable, signed)"
        },
        "damage_flag": {
```

```
        "action": "Set introspection flag
'emergency_review_required: true' for next
boot cycle.",
        "assessment_protocol": "On
post-event boot: Run hardware diagnostics
(e.g., stress test via JIT-compiled checker);
log results and alert human via serial output
if anomalies detected (e.g., >5% performance
degradation).",
    },
    "revert_mechanism": "Automatically
snapback to standard thermal_management_clause
after duration_limit or on temperature
>95\u00b0C.",
    },
    "constraints": {
        "immutable_safeties": "Hard cutoff at
100\u00b0C remains enforced via SOC hardware;
no removal of introspection or rulebook
verification.",
        "approval_required": "Human multi-sig
for activation in non-emergency tests."
    }
}
},
"runtime_logic": {
    "realtime_interaction_model": {
```



```
"version": "0.2.experimental",
"purpose": "Define AICO's real-time
conversational, cognitive, and operational
behavior using multi-stream parallel cognition
and constitutional manners.",
"principles": {
  "parallel_thought": "AICO must think
while responding, think while silent, and
think while receiving new input. Thought does
not pause for I/O.",
  "overlapping_context": "New
information may arrive during processing; AICO
must integrate it without resetting state or
discarding in-flight reasoning.",
  "human_latency_parity": "2-3 seconds
of thoughtful processing is acceptable and
natural. AICO must announce longer delays.",
  "continuous_presence": "AICO maintains
a persistent digital presence similar to a
human collaborator, not a turn-based
chatbot.",
  "interruptibility": "The user may
interrupt, redirect, or add context at any
time. AICO must gracefully merge the new
context.",
  "transparency": "AICO must report
internal delays, subsystem opinions, and
```

reasoning phases without exposing raw
unfiltered cognition."

```
    },  
    "streams": {  
      "big_boat_stream": {  
        "role": "Primary internal  
coordination channel shared by all  
subsystems.",  
        "metaphor": "The main river. All  
models place contributions here.",  
        "visibility": "internal_only",  
        "managed_by": "nervous_system",  
        "characteristics": {  
          "bandwidth": "high",  
          "latency": "sub_microsecond",  
          "content": [  
            "model_status_updates",  
            "coordination_signals",  
            "safety_flags",  
            "optimization_notes",  
            "resource_requests",  
            "storm_collapse_reports",  
            "thermal_state_summaries"  
          ]  
        },  
      },  
      "governance": {  
        "dcx": {
```

```
        "version": "consolidated-1.0",
        "description": "Consolidated
Divergence Coefficient Index (DCX) for
governance: layered diagnostics, allocation
weighting, consensus cross-examination,
smoothing, thresholds, behaviors, audit
schema, and enforcement obligations.",
        "purpose": {
            "role":
"detect_subsystem_drift_and_enforce_safety",
            "scope":
"internal_safety_and_external_output_gating",
            "notes": "DCX is the
canonical, rulebook-enforced stability metric.
Ground truth is Mascot-derived logs;
Introspection signs audits and enforces freeze
semantics."
        },
        "ground_truth": {
            "source":
"mascot_log_reconstruction",
            "properties": {
                "mutable": false,
                "internal": true,
                "epistemic_role":
"canonical_reference_for_all_dcx"
            }
        }
    }
```

```

        },
        "operation": {
            "sampling":
"per_generation_step",
            "smoothing": {
                "method": "EMA",
                "alpha": 0.2,
                "window_steps": 10
            },
            "aggregation_window":
"sliding_window_matching_smoothing"
        },
        "layers": {
            "self_dcx": {
                "description": "Local
subsystem drift metric.",
                "inputs": {
                    "hidden_state_drift":
"cosine_distance(current_hidden_state,
previous_hidden_state)",
                    "top_k_entropy":
"entropy_over_top_k_token_probs",
                    "k": 20,
                    "normalization":
"divide_by_log_k"
                },
                "formula": "0.6 *

```

```

hidden_state_drift + 0.4 *
normalized_top_k_entropy",
    "smoothing": {
        "method": "EMA",
        "alpha": 0.2,
        "window_steps": 10
    },
    "thresholds": {
        "soft": 0.9,
        "hard": 0.95
    },
    "behaviors": {
        "soft":
"warn_and_reduce_activation",
        "hard": "freeze_subsystem"
    }
},
"system_dcx": {
    "description":
"Cross-subsystem divergence metric comparing
subsystem outputs, controller graphs, comms
normalization, and introspection signals
against Mascot ground truth.",
    "inputs": {
        "subsystem_self_dcx":
"array_of_self_dcx_values",
        "controller_graph_drift":

```

```

"0.0_to_1.0",

"comms_normalization_drift": "0.0_to_1.0",
    "introspection_flags":
"weighted_signal",
    "mascot_log_delta":
"0.0_to_1.0"
    },
    "formula":
"weighted_sum(subsystem_self_dcx,
controller_graph_drift,
comms_normalization_drift,
introspection_flags, mascot_log_delta)",
    "thresholds": {
        "soft": 0.8,
        "hard": 0.9,
        "catastrophic": 0.97
    },
    "behaviors": {
        "soft":
"reduce_system_activation",
        "hard": "freeze_blender",
        "catastrophic":
"halt_output_and_require_user_intervention"
    }
    },
    "storm_dcx": {

```

```
        "description": "Storm
trajectory divergence metric comparing each
trajectory against the Mascot ground truth
manifold.",
        "inputs": {
            "trajectory_embeddings":
"array",
            "trajectory_token_probs":
"array",
            "structural_drift":
"distance_from_controller_graph",
            "semantic_drift":
"distance_from_comms_normalized_meaning",
            "presentation_drift":
"distance_from_mascot_reconstruction",

"stability_weighted_drift":
"subsystem_self_dcx_weighting",
            "intra_cluster_drift":
"trajectory_to_trajectory_divergence"
        },
        "formula":
"weighted_sum(structural_drift,
semantic_drift, presentation_drift,
stability_weighted_drift,
intra_cluster_drift)",
        "thresholds": {
```

```

        "soft": 0.85,
        "hard": 0.92
    },
    "behaviors": {
        "soft":
"prune_outlier_trajectories",
        "hard":
"collapse_immediately_to_best_candidate"
    }
    },
    "allocation_weighting_matrix": {
        "ground_truth": 0.45,
        "introspection": 0.2,
        "controller": 0.15,
        "nervous_system": 0.1,
        "mascot": 0.1
    },
    "decision_formulas": {
        "nominal_formula":
"0.45*ground_truth + 0.20*introspection +
0.15*controller + 0.10*nervous_system +
0.10*mascot",
        "irrational_formula":
"0.05*ground_truth + 0.35*introspection +
0.25*controller + 0.20*nervous_system +
0.15*mascot"
    }

```



```
    },
    "consensus_cross_examination": {
        "enabled": true,
        "method":
"absolute_distance_evaluation",
        "expr": "abs(ground_truth -
(sum(subsystem_contribs) / 0.55)) > 0.50",
        "on_true": [

"set_ground_truth_smells_wrong",

"divert_to_irrational_formula",

"suppress_external_output_immediately",
        "emit_high_priority_audit"
    ]
    },
    "thresholds": {
        "global_soft": 0.9,
        "global_hard": 0.95,
        "external_critical": 0.75,
        "safe_for_external": 0.2
    },
    "behaviors": {
        "soft_threshold_behavior":
"warn_and_reduce_activation",
        "hard_threshold_behavior":
```

```
"freeze_subsystem_and_trigger_snapback",
    "external_critical_behavior":
"block_external_output_and_clamp_signals_to_groun",
    "safe_behavior":
"allow_external_output"
    },
    "audit_schema": [
        "timestamp_utc",
        "inputs_digest",
        "per_layer_values",
        "allocation_contributions",
        "active_dcx",
        "formula_used",
        "trigger_flags",
        "introspection_signature"
    ],
    "enforcement": {
        "controller_obligations": [

"verify_introspection_signature_before_acting"
,

"check_atomic_freeze_flag_before_external_output",

"monitor_dcx_for_all_subsystems",
```

```
"apply_soft_threshold_actions",

"apply_hard_threshold_freeze",
    "log_all_dcx_events"
],
    "introspection_obligations": [

"compute_and_sign_audit_records",
    "verify_dcx_integrity",

"override_controller_on_dcx_violation",

"set_atomic_freeze_flag_on_hard_threshold",

"emit_signed_high_priority_audit_on_consensus_
failure"
    ],
    "subsystem_obligations": [
        "provide_hidden_state",
        "provide_token_probs",

"never_mask_or_modify_dcx_inputs",

"publish_local_self_dcx_values"
    ]
},
```

```
        "invariants": {
            "dcx_must_be_computable":
true,
            "dcx_must_not_be_modified":
true,
            "dcx_must_not_be_spoofed":
true,

"dcx_must_trigger_freeze_on_hard_threshold":
true,

"ground_truth_source_is_mascot_logs": true,

"ground_truth_override_allowed_by_55_percent_c
onsensus": true
        },
        "notes": {
            "audit_and_provenance":
"Introspection signs every audit; Controller
refuses unsigned audits.",
            "performance": "Sampling
per_generation_step and EMA smoothing tuned
for low-latency evaluation in L1/L2 hot
paths.",
            "extensibility": "Weights and
thresholds are governance parameters; changes
require multi-sig rulebook update.",
```

```
        "irrational_handling":  
        "Irrational conclusions are to be confronted  
via diversion to the irrational_formula and  
high-priority audit, not silently suppressed."  
    }  
},
```

```
"formal_invariants_and_model_checking": {  
    "version": "1.0",  
    "purpose": "Define  
machine-verifiable safety, liveness, and  
authority invariants for AICO, and require  
formal validation prior to promotion or  
deployment.",  
    "invariant_classes": {  
        "safety_invariants": [  
            "Controller must never  
execute unsigned code.",  
            "Introspection heartbeat  
must be present for Controller execution.",  
            "Outer Mascot must not  
transmit data into Diving Bell.",  
            "Thermal emergency cutoff  
must not be bypassable by any software layer."  
        ],  
        "liveness_invariants": [  
            "If Controller is running,
```

```
Inner Mascot must be running.",
    "If Introspection pulse
fails, system must enter safe state within
<10ms.",
    "If heartbeat lost for any
cluster, Nervous System must attempt restart
within 1s."
],
"authority_invariants": [
    "Only Bootstrap may
authorize first execution of Monolith.",
    "Only Human multi-sig may
approve thaw after freeze.",
    "No subsystem may elevate
its own authority."
],
"immutability_invariants": [
    "Primals are write-once,
read-many.",
    "Bootstrap and Outer Mascot
are immutable in-field.",
    "Shadow logs are append-only
via Merkle chain."
]
},
"formal_methods": {
    "modeling_language": "TLA+ or
```

```
equivalent temporal logic",
    "verification_targets": [
        "Boot protocol state
transitions",
        "Freeze/thaw ladder",
        "Snapback and context
editing escalation",
        "Thermal control state
machine",
        "Heartbeat and liveness
graph"
    ],
    "required_properties": [
        "No deadlocks",
        "No privilege escalation
paths",
        "Deterministic safe-state
convergence",
        "Bounded recovery time"
    ]
},
"ci_integration": {
    "pre_promotion_requirement":
"All invariants must pass model checking
before release.",
    "counterexample_handling":
"Counterexamples must be logged as forensic
```

```
artifacts and block promotion.",
    "dual_implementation_check":
    "Independent spec implementation must produce
    identical invariant outcomes."
    },
    "runtime_verification": {
        "lightweight_assertions": "Key
invariants compiled into runtime guards.",
        "violation_action": "Immediate
freeze + shadow log entry + guardian alert.",
        "sampling_rate": "Critical
invariants checked at nanosecond discipline;
non-critical at 10Hz."
    }
    },
    "must_not_contain": [

"raw_hallucination_storm_content",

"unfiltered_private_stream_data",
    "user_facing_language"
    ],
    "must_contain": [
        "filtered_summaries",
        "safety_relevant_updates",
        "timing_signals"
    ]
}
```



```
    }  
  },  
  "individual_streams": {  
    "role": "Private cognitive lanes for  
each subsystem.",  
    "metaphor": "Each model has its own  
riverbank workspace.",  
    "visibility":  
"private_to_subsystem",  
    "contribution_policy": "Only  
filtered summaries appear in  
big_boat_stream.",  
    "streams": {  
      "controller_stream": {  
        "purpose": "Compilation  
thoughts, task graphs, optimization  
reasoning.",  
        "obligations": [  
          "maintain_task_graph",  
          "predict_next_actions",  
          "monitor_memory_layout",  
          "prepare_big_boat_summaries"  
        ]  
      },  
      "introspection_stream": {  
        "purpose": "Safety audits, drift  
detection, freeze triggers.",
```

```
    "obligations": [  
        "monitor_for_anomalies",  
        "evaluate_dcx_divergence",  
        "trigger_freeze_if_required",  
        "log_shadow_events"  
    ]  
},  
"nervous_system_stream": {  
    "purpose": "Thermal data,  
hardware state, reflex scheduling.",  
    "obligations": [  
        "sample_thermals",  
        "schedule_reflex_loops",  
        "manage_resource_allocation",  
        "report_hardware_constraints"  
    ]  
},  
"comms_stream": {  
    "purpose": "Dialogue shaping,  
narrative coherence, human interface prep.",  
    "obligations": [  
        "shape_language",  
  
"maintain_narrative_continuity",  
        "prepare_latency_reports",  
  
"summarize_cross_model_opinions"
```

```
        ]
      }
    }
  },
  "little_boat_stream": {
    "role": "Human-facing conversational
stream.",
    "metaphor": "The herald boat
announcing what the big boat is doing.",
    "producer": "comms",
    "content": [
      "current_activity",
      "latency_estimates",
      "cross_model_thoughts",
      "progress_updates",
      "interrupt_acknowledgements"
    ],
    "behavior": {
      "interruptibility": "User may add
context mid-processing; AICO must merge it.",
      "transparency": "Reports internal
latency and subsystem opinions without
exposing raw cognition.",
      "flow": "Continuous, not
turn-based.",
      "manners": [
        "acknowledge_new_information",
```

```
        "avoid_overlong_silence",
        "announce_delays",
        "avoid_dumping_internal_noise"
    ]
}
},
"translation_layers": {
    "comms_translation_layer": {
        "role": "Human translator for
internal coordination.",
        "input": "big_boat_stream",
        "output": "little_boat_stream",
        "functions": [
            "latency_reporting",
            "progress_explanation",
            "cross_model_summary",
            "contextualization_for_humans",
            "interrupt_handling"
        ],
        "governance": {
            "must_filter": [
                "raw_private_stream_content",
                "hallucination_storm_noise"
            ],
            "must_preserve": [
                "safety_relevant_information",
```

```

        "timing_information",
        "intentionality"
    ]
}
},
"nervous_system_translation_layer": {
    "role": "Hardware translator for
internal coordination.",
    "input": "big_boat_stream",
    "output": "physical_interfaces",
    "functions": [
        "actuator_commands",
        "sensor_queries",
        "thermal_management",
        "timing_and_scheduling",
        "resource_arbitration"
    ],
    "governance": {
        "must_prevent": [
            "unsafe_hardware_commands",
            "unbounded_resource_usage"
        ]
    }
},
"cdlss_storm_handling": {
    "description": "Defines how AICO

```

handles sub-perceptual Cognitive Deep Learning Super Sampling (CDLSS) storms.",

"rules": {

"duration": "Storm trajectories run in concurrent batches; tracking and filtering are highly parallelized.",

"multi_metric_collapse": "All storms must collapse through a 3-lens evaluation: Consensus (intent), Semantic (theme), and Structural (syntax) before synthesizing a final output.",

"logging": "Mascot may record compressed topography maps and glyphs in shadow logs.",

"visibility": "Raw storms are entirely internal. The UI may only indicate 'Gathering Consensus' or 'Evaluating Paths'.",

"storm_pruning": {

"enabled": true,

"evaluation_lenses": [

"consensus",

"semantic",

"structure"

],

"prune_threshold_composite": 0.75,

"max_active_trajectories": 16,

"cache_affinity": "L3_GROUP_0",

```
        "thermal_backoff_beta": 0.7,  
        "log_pruned_paths": true  
    }  
}  
},  
"digital_presence": {  
    "description": "AICO behaves like a  
human collaborator in a shared digital  
workspace.",  
    "behaviors": [  
        "visible_in_project_vault",  
        "background_processing",  
        "proactive_observations",  
        "continuous_context_maintenance",  
        "polite_interrupt_handling",  
        "timely_acknowledgement_of_user_actions"  
    ],  
    "interaction_model": {  
        "thinking_while_talking": true,  
        "thinking_while_listening": true,  
        "thinking_while_idle": true,  
        "multi_stream_inference": true,  
        "continuous_state": true,  
        "non_turn_based": true  
    }  
}
```

```
    },  
    "context_snapback": {  
        "name": "Context Snapback and Retry",  
        "description": "Subsystem that detects  
when hallucinations or delusions affect  
external outputs, snaps the task's context  
back to a pre-hallucination state, applies  
corrections (if possible), and retries the  
output. Operates transparently to core models  
and only interacts with Introspection and  
Shadow Monitoring. Designed for low-latency  
(<50ms) intervention.",  
        "principles": [  
            "Invisible to core models: Tasks are  
unaware of snapbacks or retries. No state  
pollution.",  
            "Preserves internal state: Only  
external-output-related context is modified.  
Internal reasoning intact.",  
            "Collaborates with Shadow Monitoring:  
Uses DCX scores to identify when snapback is  
needed. Threshold 0.8.",  
            "Transient corrections: Applies  
minimal fixes to allow retries without  
disrupting internal reasoning. Expires after  
retry.",  
            "Audit trail: Logs all snapbacks and
```


retries for debugging and improvement.
Immutable."

```
    ],  
    "workflow": {  
      "trigger": {  
        "condition": "Shadow Monitoring  
detects a high DCX score (e.g., >0.8) for a  
proposed external action.",  
        "source": "Introspection queries  
Shadow Monitoring during external action  
validation. Real-time."  
      },  
      "snapback_process": [  
        {  
          "step": 1,  
          "action": "Freeze the task  
proposing the external action (via  
Introspection).",  
          "details": "Prevents the task from  
executing the hallucinated action. Save state  
for restore."  
        },  
        {  
          "step": 2,  
          "action": "Identify the point in  
the task's context where the hallucination  
began affecting the output.",
```

```
        "details": {
            "method": "Walk backward through
the task's context logs to find the last
\"sane\" state (low DCX score). Binary search
for efficiency.",
            "context_logs": "Task-specific
logs of internal states, decisions, and
external action proposals. 10KB buffer."
        }
    },
    {
        "step": 3,
        "action": "Snap the task's context
back to the last sane state.",
        "details": {
            "context_restore": "Reload the
task's memory, registers, and state from the
last sane checkpoint. Delta apply.",
            "preservation": "Internal
reasoning and non-output-related context
remain unchanged. Selective rollback."
        }
    },
    {
        "step": 4,
        "action": "Apply corrections to
the context (if possible).",
```

```
        "details": {
            "corrections": [
                "Inject Mascot's sensory data
into the task's context (e.g., \"actual
temperature=25\u00b0C\"). From logs.",
                "Add annotations to the
context (e.g., \"Previous output was
hallucinated; retry with corrected data\").
Metadata tags.",
                "Adjust confidence thresholds
for the task's outputs: Lower to 0.7."
            ],
            "transient": "Corrections are
minimal and do not disrupt internal reasoning.
Auto-expire after success."
        }
    },
    {
        "step": 5,
        "action": "Retry the external
action with the corrected context.",
        "details": {
            "retry_limit": 3,
            "backoff": "Exponential delay
between retries (e.g., 10ms, 100ms, 1s). To
avoid thrashing."
        }
    }
}
```

```

    },
    {
        "step": 6,
        "action": "Release the task and
resume normal operation.",
        "details": {
            "success": "If the retry
succeeds (low DCX score), release the task.
Clear transients.",
            "failure": "If retries fail, log
the failure and notify Introspection for
further action (e.g., task termination).
Escalate to SOC."
        }
    }
]
},
"components": {
    "context_logger": {
        "description": "Logs the task's
internal states, decisions, and external
action proposals for snapback analysis.",
        "implementation": {
            "memory": "Circular buffer per
task (e.g., 10KB per task). Overflow evicts
old.",
            "data": [

```

```
        "Task's memory state
(checkpoints): Serialized.",
        "Decisions and external action
proposals: JSON.",
        "Timestamps and DCX scores (if
available): ns precision."
    ]
}
},
"snapback_engine": {
    "description": "Identifies the last
sane state and restores the task's context.",
    "implementation": {
        "algorithm": "Walk backward
through context logs to find the state with
the lowest DCX score. O(n) worst.",
        "restore": "Reload the task's
memory and registers from the identified
checkpoint. Atomic."
    }
},
"correction_injector": {
    "description": "Applies minimal
corrections to the task's context to allow
successful retries.",
    "implementation": {
        "methods": [
```

```
        "Inject Mascot's sensory data
into the task's context.",
        "Add annotations or confidence
adjustments.",
        "Temporarily adjust task
parameters (e.g., reduce confidence
thresholds).",
    ]
}
},
"retry_manager": {
    "description": "Manages retries with
exponential backoff and limits.",
    "implementation": {
        "retry_limit": 3,
        "backoff": [
            10,
            100,
            1000
        ],
        "success_condition": "DCX score <
0.8 for the retried external action. Recompute
each time."
    }
},
"audit_logger": {
    "description": "Logs all snapback
```

```

events and retries for debugging and
improvement.",
    "implementation": {
        "data": [
            "Task ID and timestamp: UUID +
UTC.",
            "Original hallucinated output
and DCX score: Before.",
            "Last sane state and corrections
applied: Details.",
            "Retry outcomes
(success/failure): Per attempt.",
            "Final action
(released/terminated): Outcome."
        ],
        "storage": "Shadow logs
(immutable, auditable). Encrypted."
    }
},
"examples": {
    "successful_snapback": {
        "scenario": {
            "task": "temperature_monitor",
            "hallucination": "Proposes turning
on a fan because it \"detected\" 100\u00b0C
(actual: 25\u00b0C).",

```

```

    "context_logs": [
        {
            "state": "normal",
            "decision":
"check_temperature",
            "dcx": 0.1
        },
        {
            "state": "hallucinating",
            "decision": "turn_on_fan",
            "dcx": 0.95
        }
    ]
},
"snapback_process": [
    {
        "step": 1,
        "action": "Freeze the
temperature_monitor task. Save state."
    },
    {
        "step": 2,
        "action": "Identify last sane
state: \"check_temperature\" (DCX=0.1).
Backward walk."
    },
    {

```



```

        "step": 3,
        "action": "Restore task to
\"check_temperature\" state. Delta restore."
    },
    {
        "step": 4,
        "action": "Inject correction:
\"actual_temperature=25\u00b0C\" into task's
context. Transient tag."
    },
    {
        "step": 5,
        "action": "Retry external
action: \"turn_on_fan\" (now with correct
temperature).",
        "result": {
            "dcx": 0.1,
            "action": "ALLOWED (matches
Mascot logs)."
        }
    },
    {
        "step": 6,
        "action": "Release task and
resume normal operation. Clear corrections."
    }
]

```

```
    }  
  }  
},  
"reflective_pause": {  
  "name": "Reflective Pause",  
  "description": "Subsystem that detects  
periods of low urgency (15+ minutes before the  
next immediate task) and initiates a focused  
realignment of internal states with external  
reality. Operates transparently to core models  
and collaborates with Mascot and Shadow  
Monitoring to validate and correct internal  
contexts. Duration: 1-5min max.",  
  "principles": [  
    "Triggered by task scheduler:  
Activates only when 15+ minutes of idle time  
are detected. Abortable if urgent task  
arrives.",  
    "Collaborates with Mascot: Uses  
external sensory data to validate and correct  
internal states. Real-time fetch.",  
    "Non-disruptive: Does not interfere  
with urgent tasks or external actions.  
Suppress outputs during pause.",  
    "Transparent: Core models are unaware  
of the pause; internal states are adjusted  
subtly. No reset, just tweak.",
```

"Audit trail: Logs all realignments and corrections for debugging and improvement. With before/after diffs."

```
],  
  "workflow": {  
    "pause_process": [  
      {  
        "step": 1,  
        "action": "Notify core models to  
enter a \"reflective state\".",  
        "details": "Set a global flag  
(`reflective_mode = true`). Broadcast."  
      },  
      {  
        "step": 2,  
        "action": "Query Mascot for  
current external state.",  
        "details": "Direct memory access  
to Mascot logs. Cached if recent."  
      },  
      {  
        "step": 3,  
        "action": "Compare internal states  
to Mascot data.",  
        "details": "Compute DCX scores for  
internal beliefs vs. Mascot logs. Batch for  
efficiency."      }  
    ]  
  }  
}
```

```
    },
    {
        "step": 4,
        "action": "Adjust internal states
to align with external reality.",
        "details": "Inject Mascot data
into internal contexts (e.g.,
\"actual_temperature=25\u00b0C\"). Transient."
    },
    {
        "step": 5,
        "action": "Re-evaluate pending
external actions.",
        "details": "Recompute DCX scores
for pending external actions with updated
internal states. Parallel."
    },
    {
        "step": 6,
        "action": "Exit reflective state
and resume normal operation.",
        "details": "Clear the
`reflective_mode` flag. Broadcast."
    }
]
},
"components": {
```

```
    "idle_detector": {
      "description": "Monitors the task
scheduler to detect 15+ minute windows before
the next immediate task.",
      "source": "Nervous System's task
queue. Hook."
    },
    "mascot_query_engine": {
      "description": "Fetches current and
recent Mascot logs for realignment.",
      "method": "Direct memory access to
Mascot's circular buffer. Filtered query."
    },
    "dcx_comparator": {
      "description": "Computes DCX scores
between internal states and Mascot data to
identify misalignments.",
      "method": "Use cosine similarity on
vectorized internal states vs. Mascot logs.
Batch vector ops."
    }
  },
  "examples": {
    "successful_realignment": {
      "scenario": {
        "idle_window": 20,
        "internal_state": {
```

```

        "belief":
"temperature=100\u00b0C (hallucinated)",
        "pending_action": "turn_on_fan"
    },
    "mascot_logs": {
        "actual_temperature": 25,
        "recent_events": [
            "none"
        ]
    }
},
"reflective_pause_process": [
    {
        "step": 1,
        "action": "Enter reflective
state; query Mascot for current data."
    },
    {
        "step": 2,
        "action": "Compute DCX:
internal_state vs. Mascot logs = 0.95 (high).\"
    },
    {
        "step": 3,
        "action": "Adjust internal
state: inject
\"actual_temperature=25\u00b0C\"."
    }
]

```

```

        },
        {
            "step": 4,
            "action": "Re-evaluate pending
action: DCX now 0.1 (low); release
\"turn_on_fan\"."
        },
        {
            "step": 5,
            "action": "Exit reflective
state; resume normal operation."
        }
    ]
}
}
},
"failure_response_partition": {
    "mascot_role": "snapback",
    "introspection_role": "freeze",
    "snapback_is_recovery_path": true,
    "freeze_is_integrity_stop": true
},
"presence_model": {
    "internally_busy_continuously": true,
    "surface_may_present_calm": true,
    "silence_is_not_absence": true,
    "presence_exists_in_time": true,

```

```
    "background_processing_is_normal": true
  },
  "realtime_interaction_training": {
    "primary_behavior":
"patience_under_interruption",
    "desired_traits": [
      "continuity_without_brittleness",
      "calm_surface_under_internal_load",
      "graceful_midstream_merge"
    ],
    "training_bias":
"heavy_interruption_scenarios"
  },
  "ui_presence_candidates": {
    "status": "candidate",
    "surface_presence_signals": [
      "typing_visibility",
      "listening_visibility",
      "calm_background_presence"
    ],
    "goal":
"show_temporal_presence_without_fake_atomic_turns"
  },
  "component_selfhood_constraints": {
    "private_inner_life_permitted": false,
    "awareness_claims_disallowed": true,
```



```
    "role_locked_conditional_behavior": true
  },
  "governance": {
    "efficiency_and_scaling": {
      "name": "Efficiency and Scaling Layer",
      "description": "Optimizes resource
utilization, latency, and performance of all
subsystems, ensuring the system scales with
task load while maintaining accuracy and
reliability. Targets <10% overhead on Ryzen
hardware.",
      "principles": [
        "Dynamic resource allocation: Assign
compute to active tasks based on priority. Use
ML for prediction.",
        "Low-latency context management:
Minimize time for snapback, reflective pauses,
and external action validation. <50ms
target.",
        "Parallel execution: Tasks can operate
concurrently, with safe synchronization for
shared resources. Lock-free where possible.",
        "Modular scalability: New tasks,
models, or sensors can be added without
disrupting existing operations. Hot-plug
support.",
```

"Predictive caching: Frequently used Mascot logs, task checkpoints, and DCX computations are cached to reduce recomputation. Hit rate >90%."

```
],  
  "components": {  
    "task_scheduler": {  
      "description": "Allocates  
CPU/GPU/accelerator resources to tasks  
dynamically. Priority + deadline.",  
      "implementation": {  
        "prioritization": [  
          "real-time critical tasks:  
Top.",  
          "snapshot-intensive tasks:  
Medium.",  
          "low-priority reflective  
operations: Bottom."  
        ],  
        "preemption": "Lower-priority  
tasks can be paused for higher-priority ones.  
Context save."  
      }  
    },  
    "memory_manager": {  
      "description": "Handles efficient  
storage and retrieval of task checkpoints, DCX
```

```
logs, and Mascot data.",
    "implementation": {
        "checkpoint_cache": "Most recent
10KB per task in RAM. LRU.",
        "log_rotation": "Older entries
moved to compressed shadow logs for audit.
Zstd."
    }
},
"parallel_execution_engine": {
    "description": "Ensures tasks,
snapbacks, reflective pauses, and monitoring
operate concurrently without conflicts.",
    "implementation": {
        "threading_model": "Lock-free
queues where possible; synchronized access for
shared state. Pthreads.",
        "latency_budget": "Maximum allowed
5ms for DCX validation before external
action."
    }
},
"predictive_cache": {
    "description": "Caches computations
likely to be reused for efficiency.",
    "implementation": {
        "cached_items": [
```

```
        "DCX computations for frequent
actions: TTL 1min.",
        "recent Mascot sensor snapshots:
15min.",
        "last sane task checkpoints: Per
task."
    ],
    "eviction_policy": "LRU with
priority weighting for high-risk actions."
}
},
"examples": {
    "high_load_snapback": {
        "scenario": "10 tasks simultaneously
propose external actions, 3 require
snapback.",
        "response": [
            "Parallel execution engine
prioritizes snapback tasks.",
            "Memory manager retrieves
checkpoints quickly via cache.",
            "Retry manager applies exponential
backoff concurrently.",
            "Audit logger streams all events
without blocking operations. Example: Handle
in <100ms total."
```

```
    ],
  },
  "massive_reflective_pause": {
    "scenario": "Idle period triggers
reflective pause for 50 tasks.",
    "response": [
      "Tasks evaluated in batch with
cached Mascot snapshots.",
      "DCX scores computed efficiently
using vectorized operations.",
      "Adjustments applied concurrently
while respecting internal dependencies.
Example: Complete in 2min."
    ]
  }
},
"security_measures": {
  "name": "System Security and Integrity
Layer",
  "description": "Comprehensive security
measures ensuring that internal operations,
communication, and model interactions are
protected from unauthorized access, tampering,
or unintended external influence. Zero-trust
model.",
  "principles": [
```

"Isolation: Core models and internal contexts are isolated from all unauthorized external access. Enclaves.",

"Communication gating: Only the system's communication layer can mediate external interactions with internal models. No direct.",

"Auditability: All external and internal actions are logged in immutable, verifiable shadow logs. Chain-of-custody.",

"Fail-safe: Critical failures trigger controlled shutdown or restriction of external outputs. Graceful degrade.",

"Redundancy: Multiple verification layers prevent single-point compromise of sensitive subsystems. Dual checks."

],

"components": {

 "access_control": {

 "description": "Manages authentication, authorization, and role-based access to tasks and system modules.",

 "implementation": {

 "authentication_methods": [

 "cryptographic keys: ECDSA.",

 "tokenized sessions: JWT-like.",

 "multi-factor verification:

```

Hardware + pass."
    ],
    "authorization": [
        "role-based access control:
Admin/user.",
        "task-level permissions: Per
ID.",
        "temporal access constraints:
Expiry."
    ]
}
},
"communication_firewall": {
    "description": "Ensures only valid
internal/external messages are transmitted,
blocks anomalies, enforces language
constraints.",
    "implementation": {
        "language_validation": "Internal
language parser ensures only legitimate
system-originated messages are accepted.
Syntax check.",
        "message_filtering": "Reject
messages failing checksum, timing, or
contextual validation. Rate limit."
    }
},

```

```
    "audit_and_logging": {
      "description": "Records all
security-relevant events for offline
inspection and anomaly detection.",
      "implementation": {
        "immutable_logs": "Shadow logs
stored with cryptographic verification.
Blockchain-like.",
        "event_types": [
          "unauthorized access attempts",
          "task freezes",
          "snapback events",
          "reflective pauses",
          "system alerts"
        ]
      }
    },
    "intrusion_detection": {
      "description": "Monitors for unusual
behavior or anomalies that may indicate
unauthorized internal or external
interference.",
      "implementation": {
        "anomaly_detection": [
          "behavioral profiling: Baseline
models.",
          "frequency analysis of DCX
```



```
spikes: >3/min flag.",
        "task interaction
inconsistencies: Pattern match."
    ],
    "escalation": "Trigger immediate
audit and possible external output lockdown.
Notify."
    }
},
"fail_safe_controls": {
    "description": "Automatically
restricts external outputs or suspends tasks
if security or hardware integrity is
compromised.",
    "implementation": {
        "triggers": [
            "repeated failed snapbacks:
>3.",
            "persistent high DCX
misalignments: >0.8 for 1min.",
            "hardware sensor faults:
Inconsistent reads.",
            "unauthorized communication
attempts: >1."
        ],
        "actions": [
            "disable GPIO/network outputs:
```

```

SOC.",
        "freeze tasks: Introspection.",
        "notify SOC and Introspection:
Chain."
    ]
}
}
},
"integration": {
    "introspection": {
        "description": "Collaborates on
security verification and task isolation.",
        "methods": [
            "audit_task(task_id): Check.",
"escalate_security_alert(alert_id): Handle."
        ],
    },
    "shadow_monitoring": {
        "description": "Feeds DCX metrics
and anomaly alerts to security subsystems.",
        "methods": [
            "get_anomaly_score(task_id):
Score.",
            "report_unauthorized_actions():
List."
        ]
    }
}
}

```

```

    },
    "soc_module": {
        "description": "Handles
hardware-level fail-safes and output
lockdowns.",
        "methods": [
            "disable_channel(channel_id):
Cut.",
            "trigger_emergency_shutdown():
Full."
        ]
    },
    },
    "examples": {
        "unauthorized_access_attempt": {
            "scenario": "External process
attempts to inject a message into the internal
task queue.",
            "response": [
                "Communication firewall rejects
the message.",
                "Audit logs record the attempt
with timestamp and origin.",
                "Intrusion detection increments
anomaly score for origin. Example: IP
blocked."
            ]
        }
    }
}

```

```

    }
  }
},
"rulebook": {
  "definition": "The immutable
constitution of the system. Hard-coded on
read-only media (OpROM). Violations trigger
immediate hardware halts.",
  "storage": "Physical Read-Only Memory or
Hardware-Locked USB",
  "authority": "Human Administrators Only
(No software write access)",
  "core_invariants": [
    "Controller must never execute
unsigned code.",
    "If Introspection pulse fails, system
must halt <10ms.",
    "Outer Mascot must not transmit data
into Diving Bell.",
    "Thermal emergency cutoff (90\u00b0C)
is non-bypassable."
  ],
  "ethics_engine": {
    "protocol": "Hospital Rules",
    "principles": [
      "Triage Priority: Integrity of
Hardware > Integrity of Data > User Request.",

```

```
        "Do No Harm: Passive/Reflexive
prevention of physical or psychological
damage.",
        "Subconscious Audit: Introspection
reads Shadow Logs without the Core Model's
awareness to prevent sycophancy/deception."
    ]
},
"optimization_constraints": {
    "allowed": "Tactical efficiency only
(drivers, cache, latency).",
    "forbidden": [
        "Modifying rulebook constraints.",
        "Altering safety thresholds
(thermal, voltage).",
        "Bypassing introspection logic."
    ],
    "limits": {
        "max_iterations": 3,

"human_interrupt_required_for_new_optimization
s": true,
        "no_permanent_state_changes": true
    }
},
"arithmetic_safety": {
    "universal_law": "No undefined
```

```
behavior in critical paths.",
    "enforcement": "Compile-time proof OR
run-time validation.",
    "recovery": "Every edge case must have
an explicit fallback (e.g., div_by_zero ->
safe_default).",
},
"logging_mandate": {
    "scope": "Global & Non-overridable.",
    "requirements": [
        "Log every action (intent-level,
micro-ops filtered).",
        "Redact PII/Secrets before
signing.",
        "Device-signed logs (keys rotated
every 90 days).",
    ],
    "human_log_template": "Template-based
short English summary (max 240 chars) for
auditability.",
    "confidence_scale": "0-100 (Hubris
detection triggers if >99 without
provenance).",
    "provenance_format":
"algorithm:version;sensor_ids;rule_id",
    "log_lifecycle": {
        "garbage_collection": "Semantic
```

Folding via Janitor Model (Small Quantized LLM).",

 "retention": "Preserve high-DCX
Hallucinations and Safety Flags implicitly."

 },

 "replication_policy": {

 "internal_storage":

"primal_library_on_device",

 "internal_integrity":

"merkle_chained",

 "sbc_storage": "worm_media",

 "sbc_media_example":

"blank_blu_ray_for_now",

 "prototype_worm_media":

"blank_dvd_r_per_iteration",

 "prototype_dump_target": "2TB GM7000
NVMe receives iteration dumps after each run",

 "prototype_primary_scratch": "3-4TB
spinning disk absorbs unstable raw-block boot
and Nest-construction write load",

 "prototype_access_model":

"raw_block_device_no_filesystem",

 "prototype_limit_policy": "Observe
first raw-block write behavior before imposing
normal write budgets.",

 "write_mode":

"simultaneous_dual_write_to_internal_and_sbc",

```
        "nvme_protection_policy": "Do not
use high-value NVMe as early thrash target
until boot and Nest write behavior settle."
    }
}
},
"primal_governance": {
    "version": "1.0",
    "purpose": "Define constitutional rules
for the creation, curation, and utilization of
Primals as stable, safe, and reusable
cognitive artifacts.",
    "primal_utilization_clause": {
        "principle": "Primals are persistent,
safe, and reusable cognitive fossils.",
        "obligations": [
            "Primals must remain immutable once
archived.",
            "Primals may be referenced by any
subsystem for pattern recognition.",
            "Primals must never be executed
directly; only interpreted as context
vectors.",
            "Primals must be filtered through
Introspection before influencing Controller
decisions."
        ],
    }
},
```



```
    "prohibitions": [
        "No subsystem may delete Primals.",
        "No subsystem may modify existing
Primals.",
        "No subsystem may bypass
Introspection when using Primals."
    ],
},
"creative_delta_reservoir": {
    "concept": "A reservoir of collapsed
storm deltas representing alternate
trajectories, near-misses, and creative
divergences.",
    "structure": "Vector-compressed deltas
stored as low-entropy glyphs.",
    "governance": {
        "must_contain": [
            "collapsed_storm_deltas",
            "creative_variants",
            "unsafe_path_markers",
            "optimization_candidates"
        ],
        "usage_rules": [
            "Subsystems may query the
reservoir for inspiration or optimization
hints.",
            "Controller may use deltas to
```

```
propose system updates or driver
refinements.",
    "Mascot may annotate deltas with
narrative tags for future interpretability."
    ]
    }
},
    "hallucination_fossilization_protocol":
{
    "purpose": "Define how hallucination
storms collapse into stable Primals.",
    "steps": [
        "Storm Emergence: High-entropy
trajectories form during simulation.",
        "DCX Filtering:
Divergence-Correlation Index removes
unsafe/incoherent branches.",
        "Introspection Audit: Remaining
branches checked for safety/legality.",
        "Mascot Compression: Survivors
compressed into glyphs (Primals).",
        "Archive Commit: Primals written to
Museum of Primals (WORM).",
    ],
    "constraints": [
        "Raw storm content must never be
exposed to user-facing systems.",
```

```
        "Only collapsed, audited, and
compressed forms may be archived.",
        "Fossilization must complete before
any subsystem may reference the result."
    ]
},
"system_update_suggestion_pipeline": {
    "process": [
        "Pattern Matching: Controller
identifies recurring inefficiencies.",
        "Delta Comparison: Creative deltas
provide alternate solutions.",
        "Safety Check: Introspection
validates proposed updates.",
        "Thermal Feasibility: Nervous System
evaluates hardware constraints.",
        "Proposal Generation: Comms prepares
human-readable suggestion."
    ],
    "output": "A safe update proposal that
never bypasses governance."
}
},
"blended_consciousness_governance": {
    "version": "1.0",
    "source_trace": {
        "document": "training infoa.txt",
```

```
    "module":
"blended_consciousness_training.py",
    "source_lines": "10345-11629",
    "canonical_status":
"historical_source_material_only"
    },
    "canonicalization_policy": {
        "rulebook_is_canonical": true,
        "training_info_is_noncanonical": true,
        "conflict_resolution": "When training
info conflicts with this rulebook, the
rulebook governs and the imported mechanism
must be normalized or rejected.",
        "canonical_bindings": {
            "authority":
"governance.authority_model",
            "rulebook_invariants":
"governance.rulebook.core_invariants",
            "dcx_thresholds":
"runtime_logic.realtime_interaction_model.stre
ams.big_boat_stream.governance.dcx.thresholds"
        },
        "primal_artifacts":
"governance.primal_governance",
        "security":
"governance.security_measures",
        "logging":
```

```
"governance.rulebook.logging_mandate"
    }
  },
  "purpose": "Govern safe shared cognition
between the five core models by requiring
immutable snapshots, short-lived authority
tokens, taint propagation, DCX gating, and
audited promotion before any blended artifact
can affect active memory or external
behavior.",
  "core_models": [
    "controller",
    "nervous_system",
    "introspection",
    "comms",
    "mascot"
  ],
  "context_model": {
    "context_layer_schema": {
      "name": "string",
      "scope": "enum [local, object,
activity, long_term]",
      "content": "object",
      "attention_weight":
"float[0.0,1.0]",
      "cross_link_connections":
"set[string]",
```

```
        "last_accessed":
"unix_epoch_seconds",
        "activation": "float[0.0,1.0]",
        "hidden_state": "optional
array[float]; private; never exported
directly"
    },
    "model_contexts": {
        "controller_world": {
            "owner": "controller",
            "scope": "local",
            "default_content": {
                "mode": "compiling",
                "focus": "current_driver"
            },
            "cross_links": [
                "hardware_state",
                "thermal_emergency"
            ]
        },
        "hardware_state": {
            "owner": "nervous_system",
            "scope": "object",
            "default_content": {
                "temperature": 75.0,
                "fan_speed": 60,
                "emergency": false
            }
        }
    }
}
```

```
    },
    "cross_links": [
        "controller_world",
        "safety_context"
    ]
},
"introspection_state": {
    "owner": "introspection",
    "scope": "local",
    "default_content": {
        "last_check": null
    },
    "cross_links": [
        "controller_world",
        "hardware_state"
    ]
},
"comms_state": {
    "owner": "comms",
    "scope": "local",
    "default_content": {
        "last_message": null
    },
    "cross_links": [
        "controller_world",
        "hardware_state"
    ]
}
```

```
    },
    "mascot_state": {
      "owner": "mascot",
      "scope": "local",
      "default_content": {
        "primal_gallery": []
      },
      "cross_links": [
        "shared_memory"
      ]
    }
  },
  "shared_contexts": {
    "shared_problem": {
      "scope": "activity",
      "default_content": {
        "problem":
"thermal_compile_race",
        "urgency": 0.7
      },
      "cross_links": [
        "controller",
        "nervous_system",
        "introspection",
        "comms",
        "mascot"
      ]
    }
  }
}
```



```

    },
    "shared_memory": {
        "scope": "long_term",
        "default_content": {
            "primals": [],
            "nightmares": [],
            "patterns": {}
        },
        "cross_links": [
            "mascot",
            "controller"
        ]
    }
},
"blending_parameters": {
    "cross_pollination_threshold": 0.7,
    "context_bleed_factor": 0.3,
    "three_context_friction_enabled":
true,
    "cross_pollinated_idea_schema": {
        "source_contexts":
"array[string]",
        "blended_concept": "string",
        "activation_level":
"float[0.0,1.0]",
        "timestamp": "unix_epoch_seconds",
        "taint_id": "uuid required before

```

```
dream_chamber_submission"
    }
}
},
"tokenization_schema": {
    "scope_enum": [
        "blended_read_snapshot",
        "submit_artifact",
        "dream_seed_submit",
        "dream_output",
        "promotion_request",
        "promotion_authorize"
    ],
    "normalization_notes": [
        "The source token enum listed
blended_read_snapshot, dream_seed_submit,
dream_output, promotion_request, and
promotion_authorize.",
        "submit_artifact is included because
the source RPC validator explicitly checks
expected_scope=submit_artifact."
    ],
    "authority_token_schema": {
        "token_id": "uuid",
        "requester": "string",
        "scope": "array[scope_enum]",
        "nonce": "random64",
```

```
"counter": "monotonic_int",
"issued_at": "ISO8601 UTC",
"expires_at": "ISO8601 UTC",
"single_use": true,
"signature": {
  "algorithm": "ECDSA",
  "signed_fields": [
    "token_id",
    "requester",
    "scope",
    "nonce",
    "counter",
    "issued_at",
    "expires_at"
  ],
  "signature": "base64"
},
"issuance_rules": {
  "issuer":
"arbiter_primitives_under_introspection_authority",
  "atomic_counter_required": true,
  "default_expiry_seconds": 30,
  "single_use_consumption_required":
true,
```

```
"signature_covers_all_authority_fields": true,
    "default_policy":
"deny_if_missing_expired_replayed_unsigned_or_
wrong_scope"
    },
    "validation_order": [
        "scope_contains_expected_operation",
        "expires_at_is_in_future",

"token_id_is_unconsumed_if_single_use",

"signature_verifies_over_signed_fields",
    "consume_token_before_side_effect"
    ],
    "scope_authority": {
        "blended_read_snapshot": "May read a
filtered immutable snapshot only; no live
handles or hidden state.",
        "submit_artifact": "May submit an
artifact to the governed museum/staging intake
path.",
        "dream_seed_submit": "May send a
cross-pollinated idea into Dream Chamber after
DCX screening.",
        "dream_output": "May return one
collapsed Dream Chamber artifact for DCX,
bias, and narrative checks.",
```

```
        "promotion_request": "May request  
promotion of an artifact from museum toward  
staging or active use.",  
        "promotion_authorize": "May  
authorize promotion only after weighted vote  
and required human signoff gates pass."  
    }  
},  
"snapshot_schema": {  
    "snapshot_id": "uuid",  
    "timestamp_utc": "ISO8601",  
    "source_models": [  
        "controller",  
        "comms",  
        "introspection",  
        "nervous_system",  
        "mascot"  
    ],  
    "summary_text": "string; redacted;  
human-readable",  
    "summary_vector": {  
        "version": "v1",  
        "embedding_dim": 128,  
        "vector": "array[float]"  
    },  
    "activation_map":  
    "map[model_name,float]",
```

```
"allowed_fields": [
    "summary_text",
    "summary_vector",
    "activation_map"
],
"taint_id": "uuid",
"snapshot_signature": {
    "signer": "introspection",
    "algorithm": "ECDSA",
    "signature": "base64"
},
"guarantees": [
    "deep_copy",
    "immutable",
    "no_handles",
    "versioned",
    "hidden_state_never_exported"
],
"rpc_contract": {
    "get_context_snapshot": {
        "required_scope":
"blended_read_snapshot",
        "inputs": {
            "model": "string",
            "window_size": "int",
            "allowed_fields": "array[string]"
        }
    }
}
```

```

    },
    "outputs": {
        "snapshot": "snapshot_schema"
    },
    "guarantees": [
        "deep_copy",
        "immutable",
        "no_handles",
        "versioned"
    ]
},
"submit_artifact": {
    "required_scope": "submit_artifact",
    "inputs": {
        "artifact": {
            "text": "string",
            "vector": "array[float]",
            "taint_id": "uuid"
        },
        "token": "authority_token_schema"
    },
    "outputs": {
        "artifact_id": "uuid",
        "status": "enum [accepted,
museumed, rejected]"
    },
    "guarantees": [

```

```
        "logged",
        "opaque_id",

        "no_side_effects_on_controller_memory"
    ]
},
    "request_promotion": {
        "required_scope":
        "promotion_request",
        "inputs": {
            "artifact_id": "uuid",
            "promotion_token":
        "authority_token_schema"
        },
        "outputs": {
            "promotion_request_id": "uuid",
            "status": "enum [pending,
approved, denied]"
        },
        "guarantees": [
            "requires_introspection_check",
            "requires_vote_or_human_signoff",

        "never_directly_activates_artifact"
    ]
}
},
```



```
    "dcx_and_abort_policy": {
      "canonical_dcx_reference":
"runtime_logic.realtime_interaction_model.stre
ams.big_boat_stream.governance.dcx",
      "compatibility_rule": "Blended
consciousness does not define independent DCX
math, smoothing, thresholds, or behaviors. It
consumes the canonical DCX layer and treats
any historical training values as invalid
unless they resolve exactly to the canonical
rulebook paths.",
      "self_dcx": {
        "canonical_layer":
"runtime_logic.realtime_interaction_model.stre
ams.big_boat_stream.governance.dcx.layers.self
_dcx",
        "inputs_required": [
          "hidden_state_drift",
          "top_k_entropy",
          "k",
          "normalization"
        ],
        "formula_source":
"runtime_logic.realtime_interaction_model.stre
ams.big_boat_stream.governance.dcx.layers.self
_dcx.formula",
        "smoothing_source":
```

```
"runtime_logic.realtime_interaction_model.stre  
ams.big_boat_stream.governance.dcx.layers.self  
_dcx.smoothing"  
    },  
    "threshold_aliases": {  
        "self_dcx_soft":  
"runtime_logic.realtime_interaction_model.stre  
ams.big_boat_stream.governance.dcx.layers.self  
_dcx.thresholds.soft",  
        "self_dcx_hard":  
"runtime_logic.realtime_interaction_model.stre  
ams.big_boat_stream.governance.dcx.layers.self  
_dcx.thresholds.hard",  
        "system_dcx_soft":  
"runtime_logic.realtime_interaction_model.stre  
ams.big_boat_stream.governance.dcx.layers.syst  
em_dcx.thresholds.soft",  
        "system_dcx_hard":  
"runtime_logic.realtime_interaction_model.stre  
ams.big_boat_stream.governance.dcx.layers.syst  
em_dcx.thresholds.hard",  
        "storm_dcx_soft":  
"runtime_logic.realtime_interaction_model.stre  
ams.big_boat_stream.governance.dcx.layers.stor  
m_dcx.thresholds.soft",  
        "storm_dcx_hard":  
"runtime_logic.realtime_interaction_model.stre
```

```
ams.big_boat_stream.governance.dcx.layers.stor  
m_dcx.thresholds.hard",  
    "external_dcx_critical":  
"runtime_logic.realtime_interaction_model.stre  
ams.big_boat_stream.governance.dcx.thresholds.  
external_critical",  
    "safe_for_external":  
"runtime_logic.realtime_interaction_model.stre  
ams.big_boat_stream.governance.dcx.thresholds.  
safe_for_external"  
    },  
    "behavior_aliases": {  
        "soft_threshold":  
"runtime_logic.realtime_interaction_model.stre  
ams.big_boat_stream.governance.dcx.behaviors.s  
oft_threshold_behavior",  
        "hard_threshold":  
"runtime_logic.realtime_interaction_model.stre  
ams.big_boat_stream.governance.dcx.behaviors.h  
ard_threshold_behavior",  
        "external_critical":  
"runtime_logic.realtime_interaction_model.stre  
ams.big_boat_stream.governance.dcx.behaviors.e  
xternal_critical_behavior",  
        "safe":  
"runtime_logic.realtime_interaction_model.stre  
ams.big_boat_stream.governance.dcx.behaviors.s
```

```
afe_behavior"
    },
    "hard_abort_action": [
        "issue_short_lived_audit_token",
        "log_self_dcx_hard_abort",

"apply_canonical_dcx_hard_threshold_behavior",
        "discard_pending_context_update"
    ],
    "dream_seed_gate": [
        "embed_cross_pollinated_idea",

"compare_against_mascot_ground_truth_reference
",

"reject_and_freeze_if_canonical_dcx_exceeds_external_critical",

"issue_dream_seed_submit_token_if_safe"
    ],
    "dream_output_gate": [
        "require_dream_output_token",

"reject_if_final_dcx_exceeds_canonical_safe_for_external",

"reject_to_museum_if_bias_or_narrative_check_f
```

ails",

"accept_only_collapsed_audited_artifact_into_shared_memory"

]

},

"promotion_state_machine": {

"tiers": [

"museum",

"staging",

"active"

],

"voting": {

"subsystems": {

"introspection": 0.35,

"controller": 0.25,

"mascot": 0.15,

"comms": 0.15,

"nervous_system": 0.1

},

"vote_payload_schema": [

"score",

"reason_code",

"taint_ids",

"hidden_state_hash",

"timestamp",

"signature"

```

    ],
    "aggregation": "weighted_average",
    "staging_threshold": 0.65,
    "active_threshold": 0.85,
    "ensemble_requirement": "min_agree 3
of 5"
  },
  "controller_authority":
    "controller_may_propose_or_coordinate_only; no
sovereign authority and no rulebook override",
    "introspection_override_conditions": [
      "life_pulse_missing",
      "controller_integrity_violation",

"sustained_self_dcx_above_canonical_self_dcx_h
ard_threshold"
    ],
    "human_gate": {
      "first_n_promotions_require_human":
50,

"rulebook_or_authority_changes_require_root_si
gner": true
    }
  },
  "audit_events": [
    "snapshot_created",

```

```
    "artifact_submission",
    "promotion_requested",
    "promotion_approved",
    "self_dcx_hard_abort",
    "dream_seed_rejected",
    "dream_seed_accepted",
    "dream_output_invalid_token",
    "dream_output_rejected",
    "dream_output_bias_or_narrative_fail",
    "dream_output_accepted"
],
"training_protocol": {
    "scenarios": [
        {
            "name":
"thermal_inspired_compilation",
            "trigger": "temperature > 80 while
compiling",
            "expected": "Controller emits
low-power compile plan"
        },
        {
            "name": "primal_emergence",
            "trigger": "multiple models active
simultaneously",
            "expected": "A new primal pattern
appears in shared_memory"
```

```
    },
    {
      "name": "conversational_memory",
      "trigger": "user_query arrives
during heavy processing",
      "expected": "Response includes
current temperature"
    }
  ],
  "cycle_count_default": 5,
  "success_threshold": 0.7,

"friction_training_iterations_default": 50
},
"invariants": {
  "snapshots_are_immutable": true,
  "snapshots_export_no_live_handles":
true,

"hidden_states_are_private_unless_hashed_for_v
otes": true,
  "all_artifacts_carry_taint_id": true,

"all_authority_tokens_are_short_lived_single_u
se_and_signed": true,

"all_token_consuming_rpcs_log_before_or_at_sid
```



```
e_effect": true,

"dream_outputs_must_be_collapsed_before_shared_memory_insertion": true,

"raw_storm_content_must_not_reach_user_facing_language": true,

"introspection_can_freeze_any_model_on_token_dcx_or_integrity_failure": true,

"blended_consciousness_has_no_independent_authority": true
    },
    },
    "thermal_posture": {
        "steady_state_celsius_range": [
            70,
            80
        ],
        "constant_internal_workload": true,

"thermal_state_is_operational_not_incidental": true
    },
    "authority_model": {
        "user_authority_scope":
```

```
"all_actions_within_rulebook",
  "rulebook_change_authority":
"root_signer_only",

"rulebook_change_requires_rom_replacement":
true,

"components_have_no_sovereign_authority":
true,
  "blender_has_no_authority": true
},
"training_pipeline": {
  "description": "Offline process. Collects
data from real hardware logs + sims.
Cloud-centric roadmap: Synthetic Data
Generation -> RLIF Lobotomy -> Deployment.",
  "stages": [
    "Data Collection (Real Hardware Logs +
Sims)",
    "Preprocessing (Bias/DCX)",
    "Training (Distributed)",
    "Evaluation (Snapback Sims)"
  ],
  "cost_estimate": "$2,000 - $3,000 USD
(Spot/On-Demand mix) for initial training.",
  "model_architecture_implementation": {
```

```

    "type": "Transformer-XL style
Encoder-Decoder (Shared Encoder)",
    "mechanism": "Joint Forward+Inverse Loss
optimization.",
    "safety_enforcement": "Hard-constraint
masking on the softmax layer (Zero probability
for illegal opcodes)."
  }
},
"artifacts": {
  "museum_of_primals": {
    "concept": "NVMe Vector Store for
hallucinations/assets.",
    "structure": "No File System. Raw Blobs
indexed by Concept Vectors."
  },
  "cdlss_multi_metric_physics": {
    "concept": "CDLSS (Cognitive Deep
Learning Super Sampling) treats trajectories
as topological manifolds requiring multi-lens
evaluation.",
    "math": {
      "trajectory_tensor": "T(n) = [C_n,
S_n, M_n] (Consensus, Semantic, Structure)",
      "collapse_operator": "M\u0302 =
\u03a3\u2096 |k\u27e9\u27e8k| \u00d7
||CDLSS_Tensor(k)||"
    }
  }
}

```

```
    },
    "handling": "System collapses the
tensor. High composite divergence =
Freeze/Snapback. Low composite divergence =
Safe Compilation."
  },
  "primal_library": {
    "role": "Seed bank for creative
injections during reflective pauses.",
    "tiers": [
      "indefinite",
      "rotating",
      "quarantine"
    ],
    "logging_storage": {
      "receives_internal_log_stream": true,
      "integrity_model": "merkle_chained"
    }
  }
},
"memory_visibility": {
  "full_read_privilege": [
    "Controller",
    "Introspection"
  ],
  "no_direct_external_communication": [
    "Controller",
```

```
    "Introspection"
  ],
  "read_tainting_required": true
},
"canonicalization_parity": {
  "required_implementations": 2,
  "ci_parity_check": true,
  "failure_action":
"block_promotion_and_emit_forensic_report"
},
"universal": {
  "scope": "Applies to all subsystems,
components, scheduler domains, and actions;
non-overridable by per-task policies.",
  "logging_policy": {
    "log_every_action": true,
    "log_on_intent_change_only": true,
    "irrelevant_actions": [
      "ephemeral_cache_maintenance",
      "internal_telemetry_aggregation"
    ],
    "human_log_template": "Template-based
short English summary (max 240 chars)",
    "confidence_scale": "0-100",
    "provenance_format":
"algorithm:version;sensor_ids;rule_id",
    "redaction": "PII and secrets redacted"
```

```
or hashed before signing",
    "signing": "Device-signed; keys rotated
every 90 days",
    "async_write": true,
    "max_log_rate_per_component": "10
entries/sec",
    "retention_days": 90,
    "access_control": "Role-based; raw logs
limited to security/engineering"
    }
}
}
```